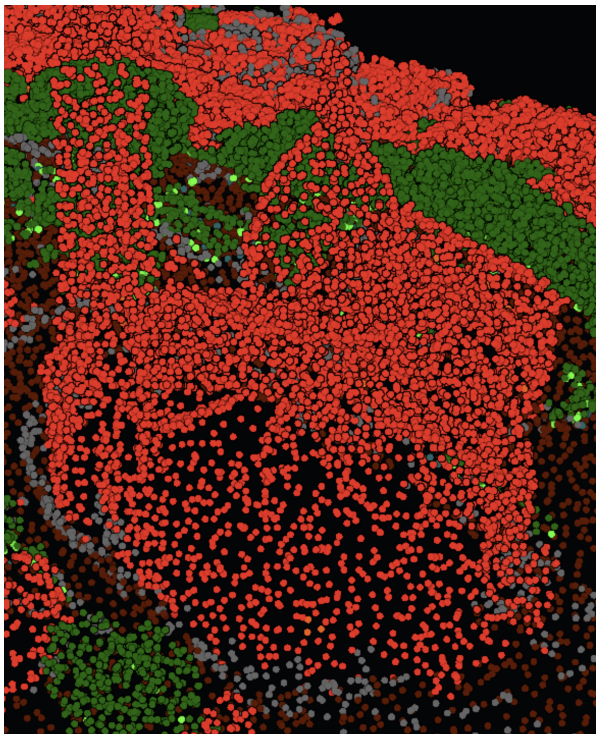
Représentation d'une cartographie par triangulation de Delaunay

Wally CHAOUI
Numéro candidat - 49398

I. Présentation du problème

Comment reconstituer une surface topographique à partir d'un ensemble fini de points ?



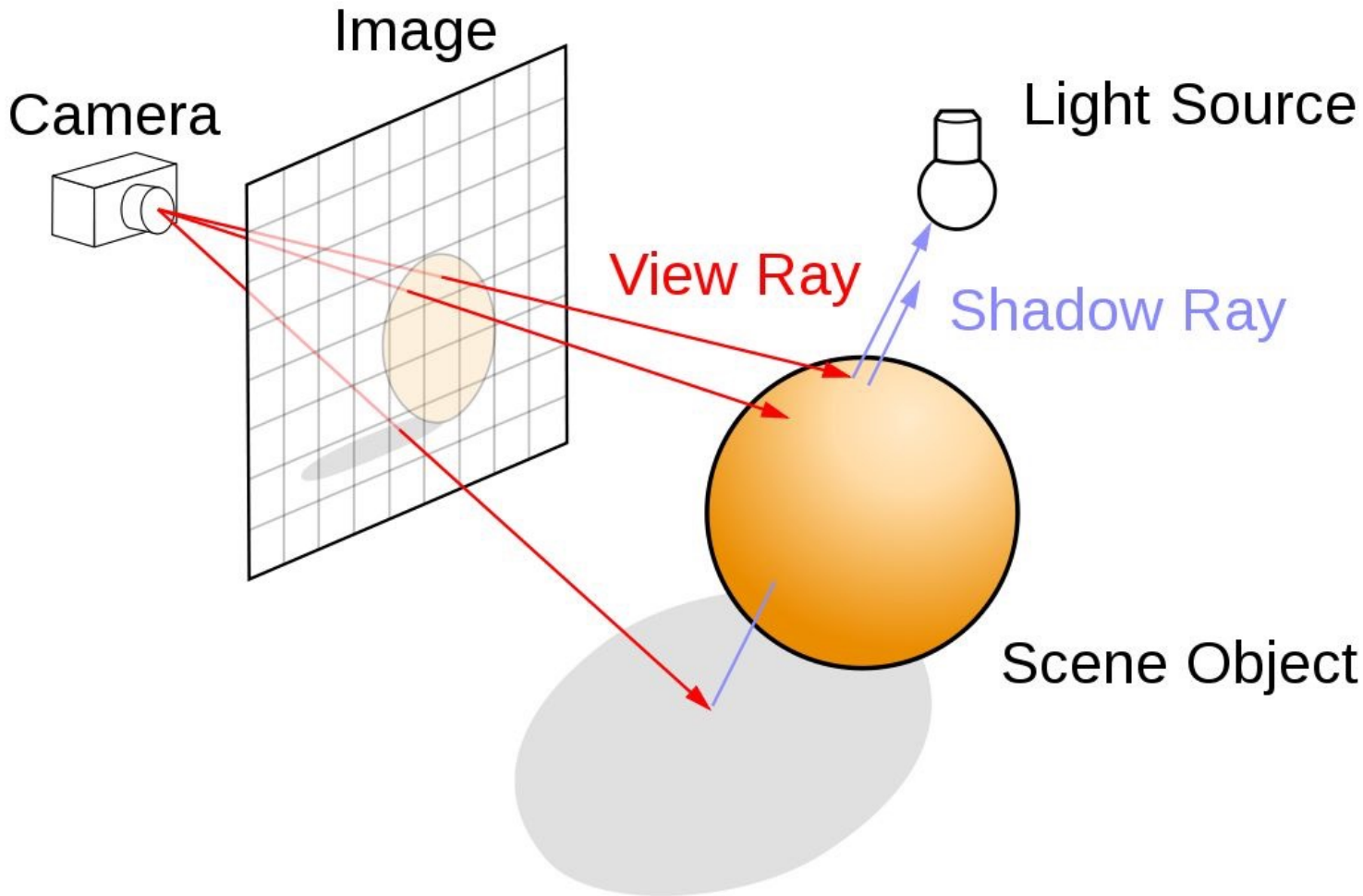
Nuage de points dans l'espace (IGN)



Maillage (Google Earth)

II. Rendus 3D et lancer de rayons

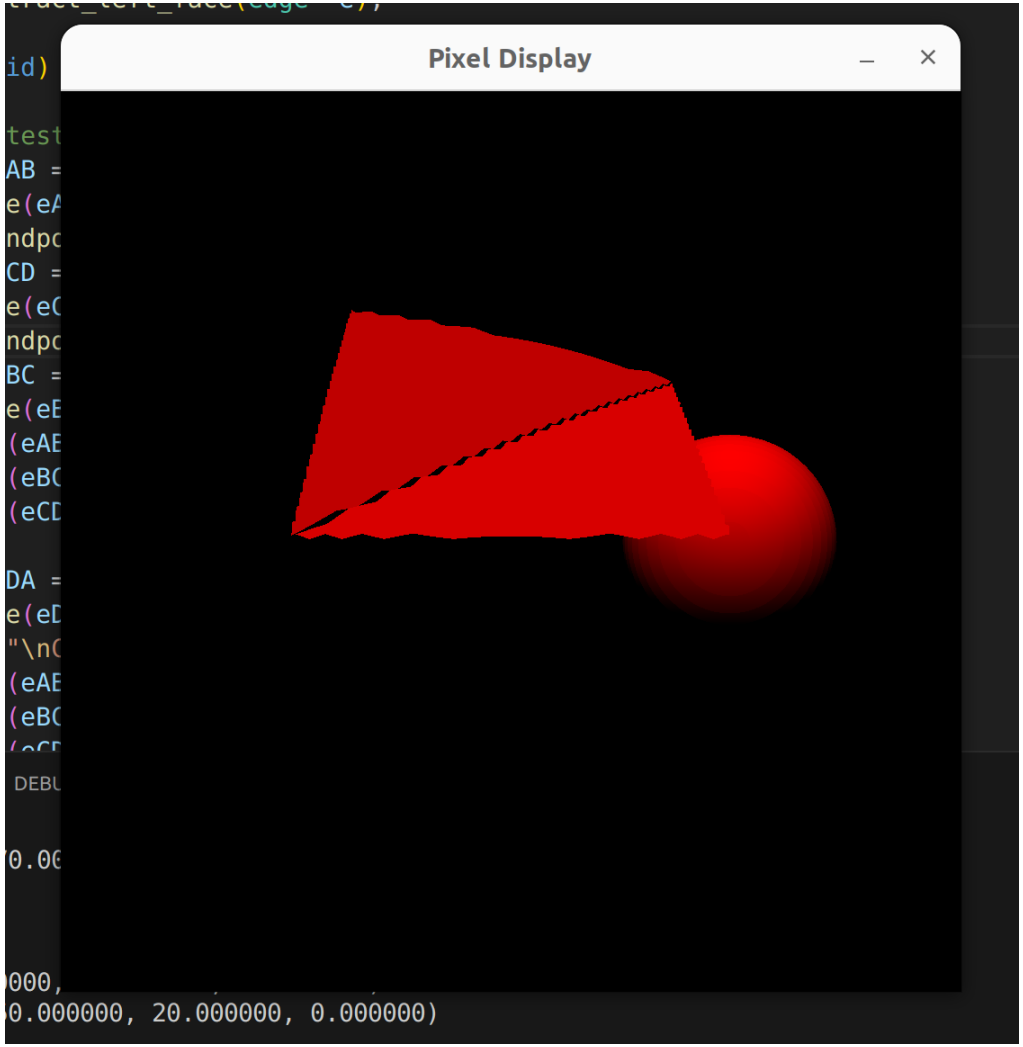
Principe



Fonctionnement du lancer de rayons (Nvidia)

II. Rendus 3D et lancer de rayons

Un premier rendu

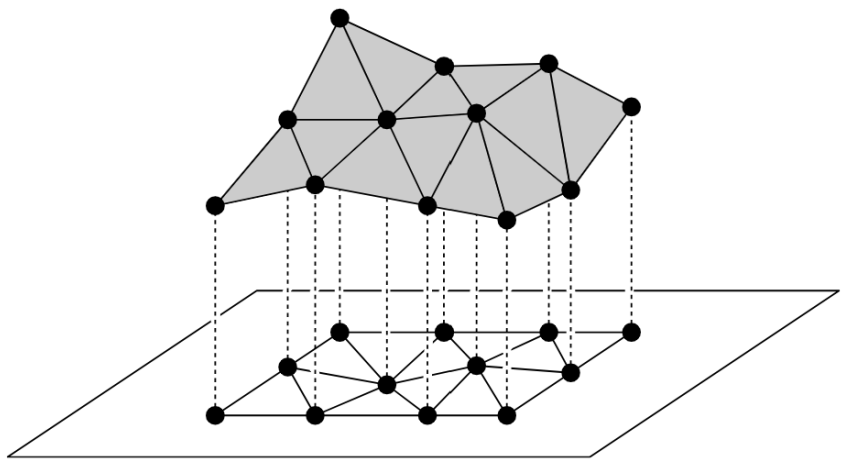


III. Triangulations

Formaliser le problème

On considère :

- $A \subset \mathbb{R}^3$ une surface
- $P \subset A$ un ensemble fini de points
- $\tilde{P} = \{(x, y)\}_{(x,y,z)\in P}$ la projection des points sur un plan



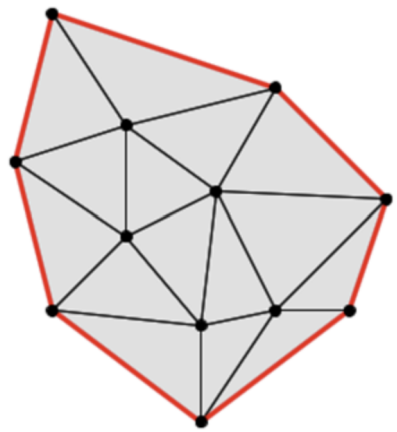
Projection des points sur le plan $z = 0$ (De Berg et al.)

III. Triangulations

Définition d'une triangulation

$T = t_1, ..., t_k$ un ensemble de k triangles tels que :

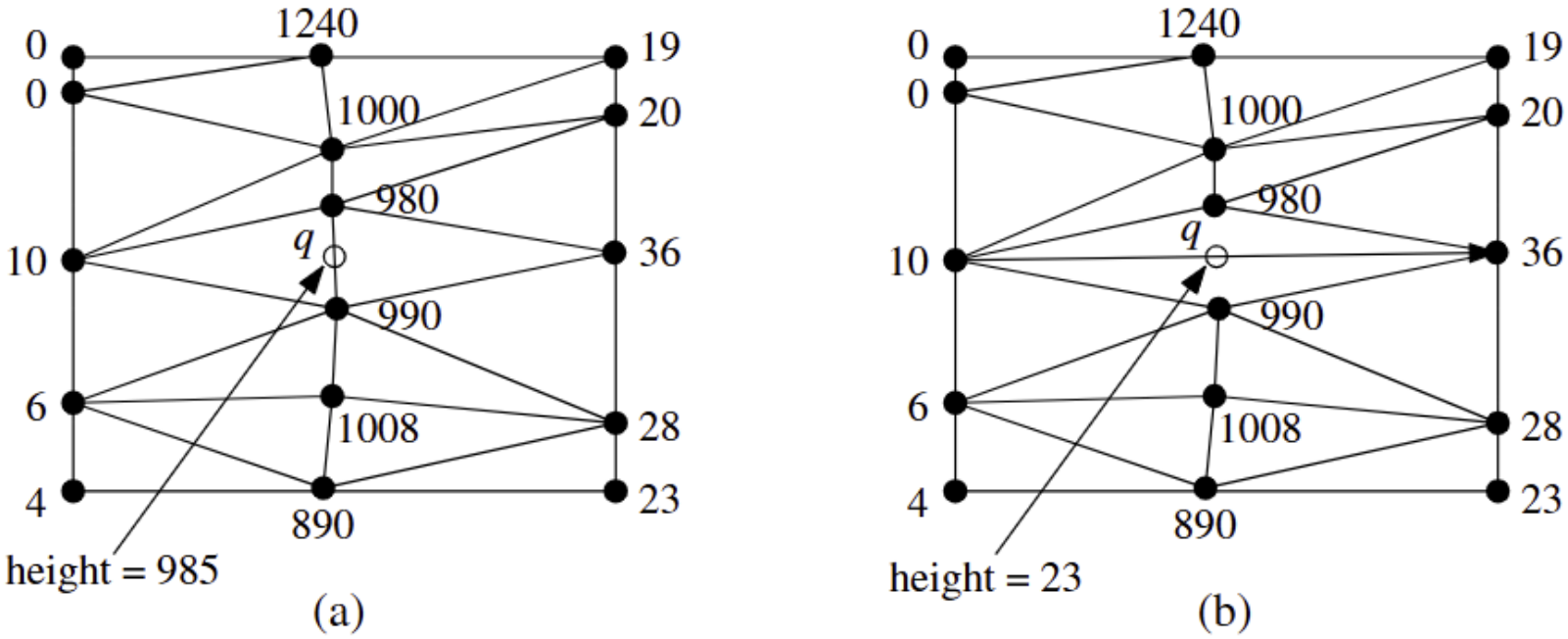
- Tous les sommets des triangles sont des points de $\tilde{P}$
- Les triangles ne se coupent pas : $\bigcap_{i=1}^{k} t_k$ est d'intérieur vide
- Maximalité : l'ajout d'un triangle doit nier cette condition



Exemple d'une triangulation (G. W. Lucas)

III. Triangulations

Intérêt d'une triangulation bien choisie



Deux triangulations (De Berg et al.)

On considère :

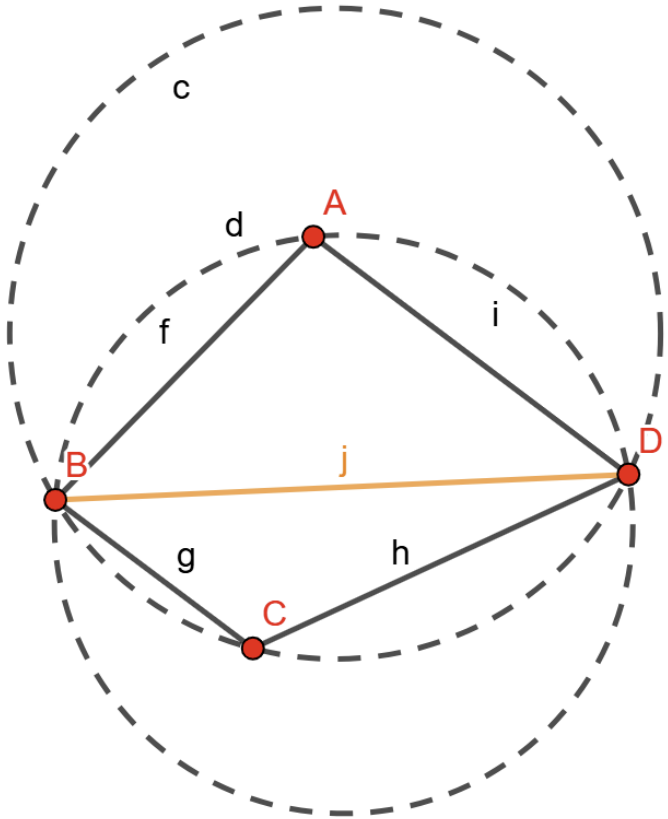
- T une triangulation
- $\{\theta_1, \ \dots \ , \theta_m\}$ l'ensemble des angles formés par ses triangles
- $\phi(T) = \min\{\theta_1, \ \dots \ , \theta_m\}$ le plus petit angle

Définition

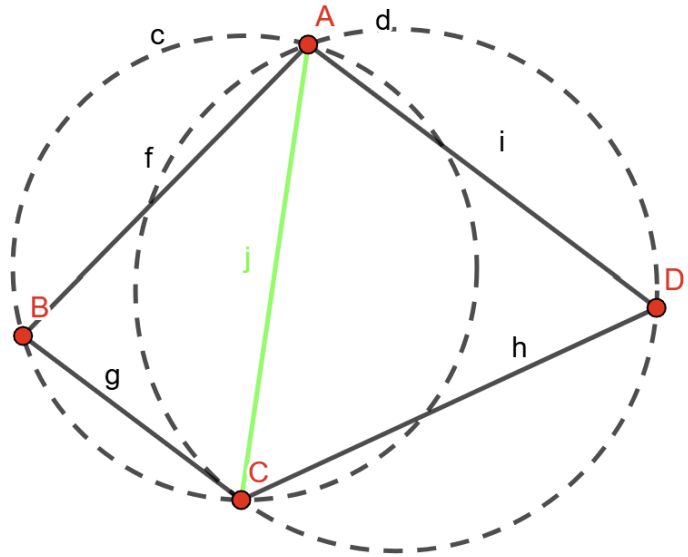
T est dite « angle-optimale » si $\phi(T)$ est maximal parmi toutes les triangulations possibles.

III. Triangulations

Obtenir l'angle-optimalité



Arête « illégale »



Arête « légale »

Définition

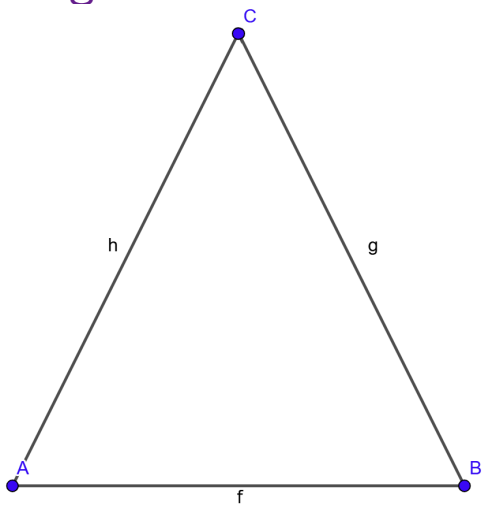
Une triangulation est dite « de Delaunay » si toutes ses arêtes sont légales.

Théorème

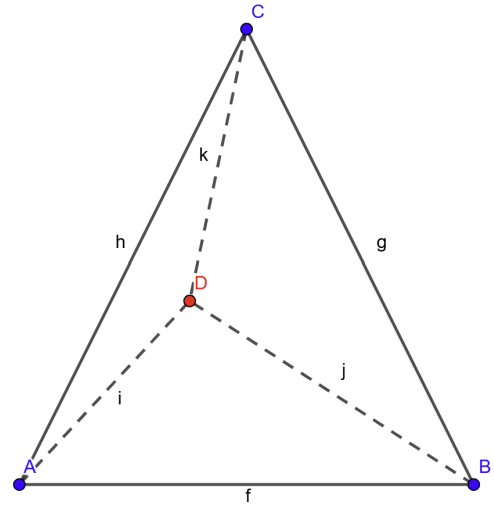
Une triangulation est angle-optimale si et seulement si c'est une triangulation de Delaunay.

III. Triangulations

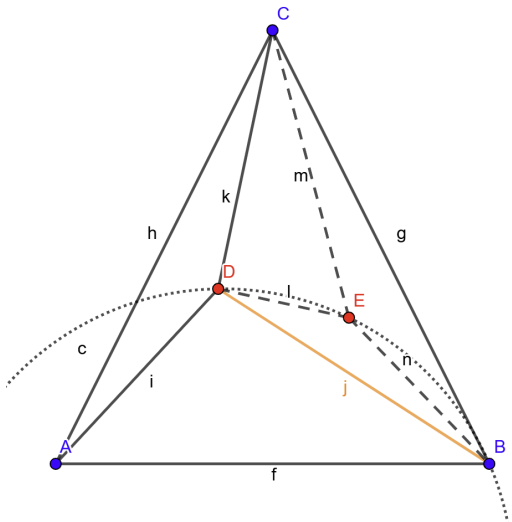
Principe de l'algorithme



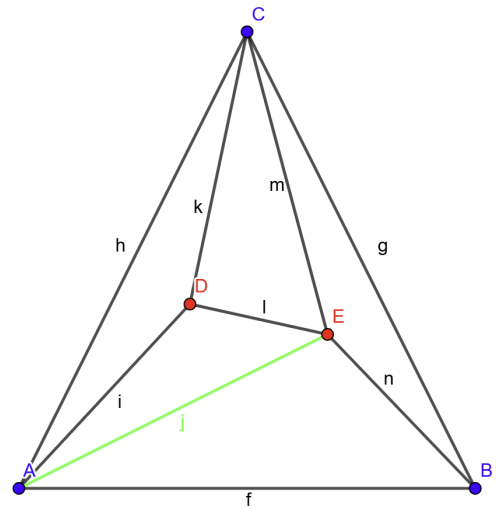
Etape 1



Etape 2



Etape 3



Etape 4

Algorithme 1: Algorithme incrémental

Entrée: Tableau P de n points

Sortie: Une triangulation de Delaunay T

Initialiser a, b, c;

$T \leftarrow T \cup \{\overline{ab}, \overline{bc}, \overline{ca}\}$;

Pour $i \leftarrow 1$ **à** n **faire**

  p_j, p_k, $p_l \leftarrow$ Localiser(p_i);

  $T \leftarrow T \cup \{\overline{p_i p_j}, \overline{p_i p_k}, \overline{p_i p_l}\}$ dans T;

  Legaliser($\overline{p_i p_j}$);

  Legaliser($\overline{p_i p_k}$);

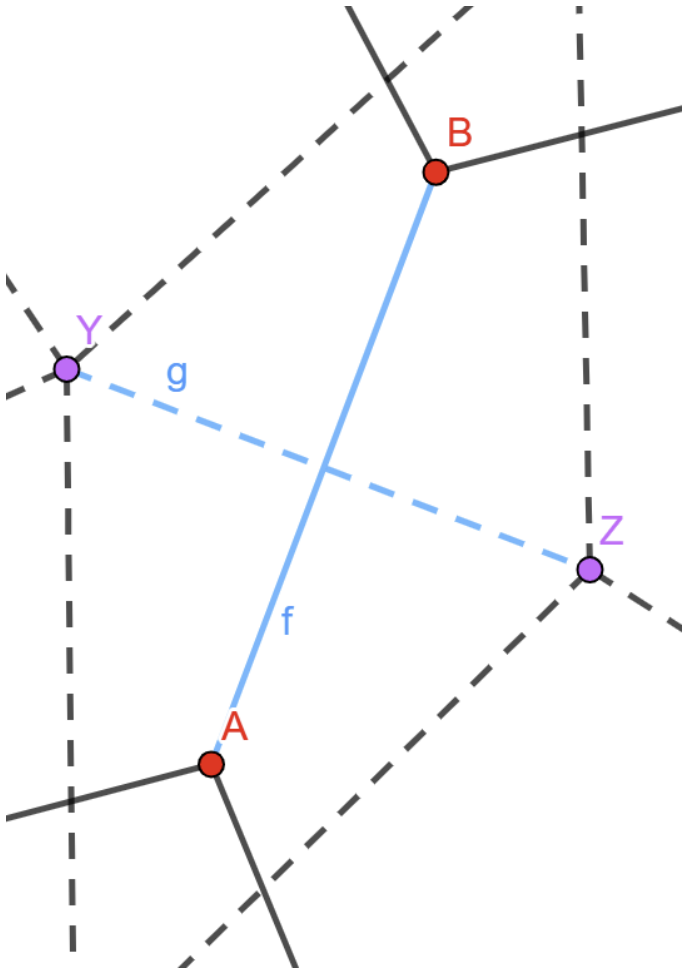
  Legaliser($\overline{p_i p_l}$);

fin

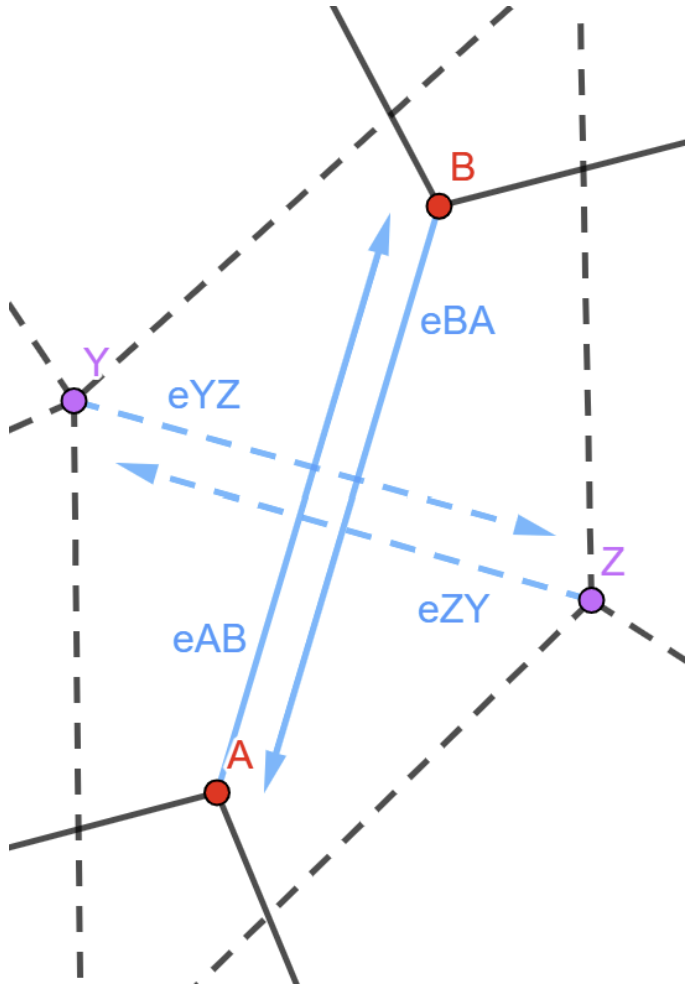
Renvoyer T;

IV. Implémentation

Le quad-edge



Une arête et son dual



Représentation en machine

IV. Implémentation

Structure de donnée

edge.h

```
1   #include "vector3.h"
2
3   struct Edge {
4       struct Edge* onext;
5       struct Edge* rot;
6       vector3 origin;
7       int data;
8   };
9   typedef struct Edge edge;
10
11  edge* e_onext(edge* e);
12  edge* e_rot(edge* e);
13  edge* e_sym(edge* e);
14  // (...)
15  edge* e_make_edge();
16  void e_splice(edge* a, edge* b);
17  edge* e_connect(edge* a, edge* b);
18  void e_remove(edge* e);
19  void e_swap(edge* e);
```

IV. Implémentation

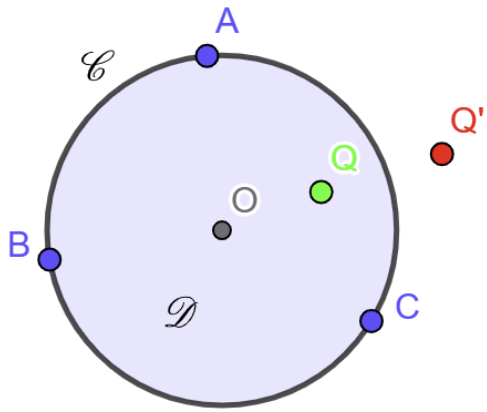
Fonction InCircle

Théorème

$$Q \in \mathscr{D} \iff \begin{vmatrix} x_A & y_A & x_A^2 + y_A^2 & 1 \\ x_B & y_B & x_B^2 + y_B^2 & 1 \\ x_C & y_C & x_C^2 + y_C^2 & 1 \\ x_Q & y_Q & x_Q^2 + y_Q^2 & 1 \end{vmatrix} \geq 0$$

Lemme

$$Q \in \mathscr{C} \iff \begin{vmatrix} x_A & y_A & x_A^2 + y_A^2 & 1 \\ x_B & y_B & x_B^2 + y_B^2 & 1 \\ x_C & y_C & x_C^2 + y_C^2 & 1 \\ x_Q & y_Q & x_Q^2 + y_Q^2 & 1 \end{vmatrix} = 0$$



IV. Implémentation

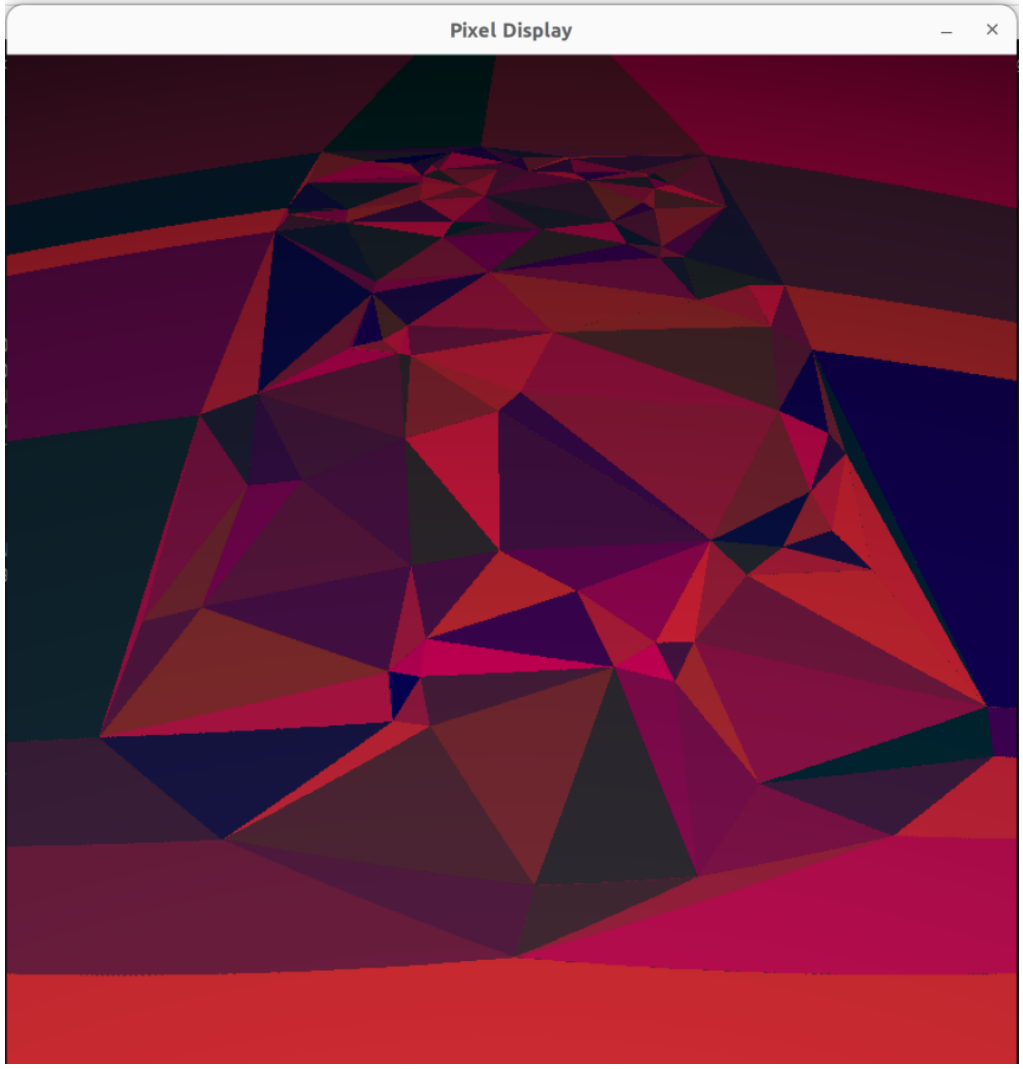
Fonctions de triangulation

delaunay2.h

```
1   #include "vector3.h"
2   #include "edge.h"
3   #include "triangle.h"
4
5   bool d2_in_circle(vector3 a, vector3 b, vector3 c, vector3 p);
6   bool d2_seen_node(edge* e);
7   double d2_triangle_area(vector3 a, vector3 b, vector3 c);
8   bool d2_ccw(vector3 a, vector3 b, vector3 c);
9   bool d2_left_of(vector3 p, edge* e);
10  edge* d2_locate(vector3 p, edge* e0);
11  void d2_insert(vector3 p, edge* e0);
12  triangle d2_extract_dual_face(edge* e);
13  int d2_extract_triangles(triangle* triangles, edge* e0);
```

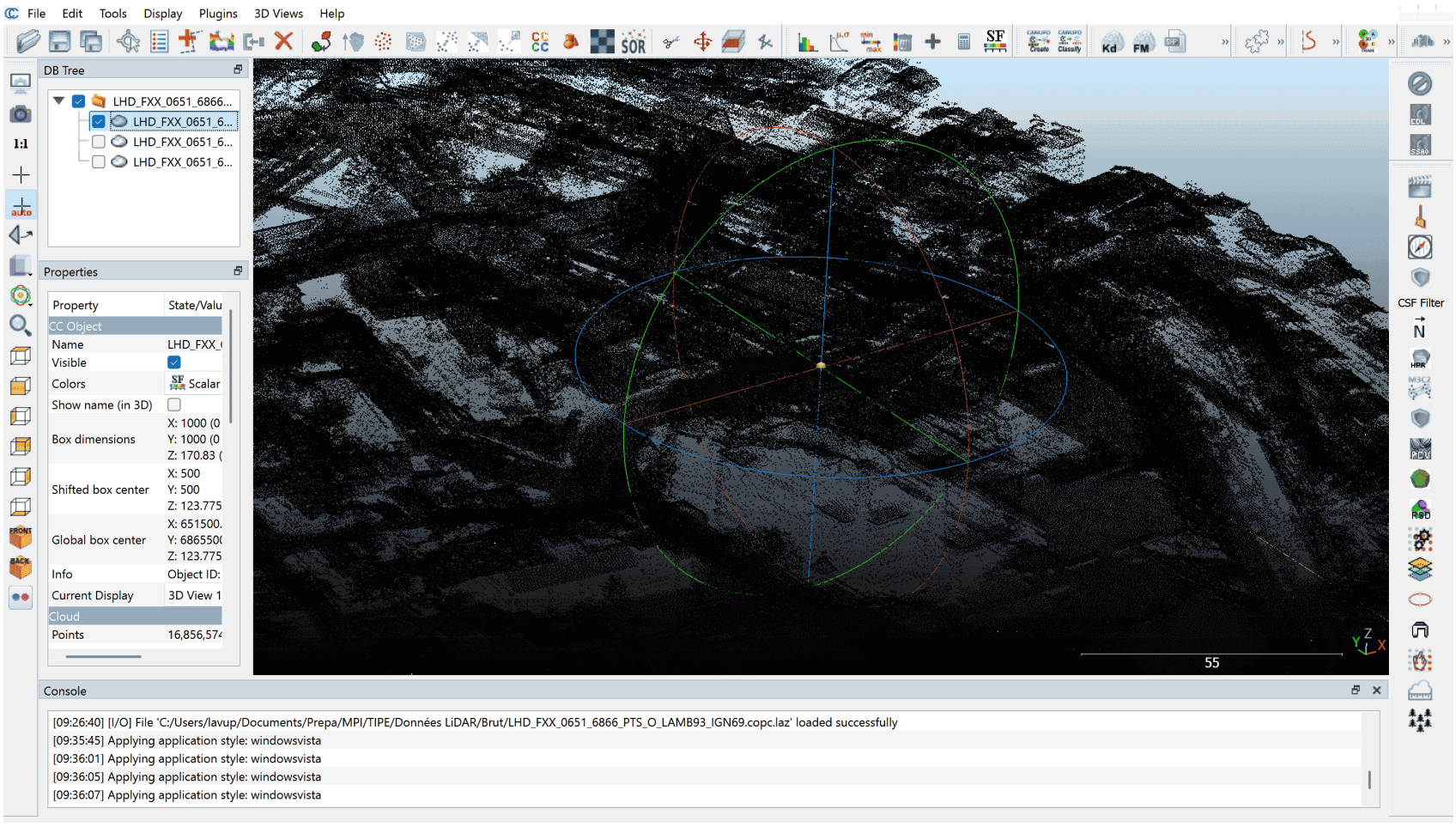
IV. Implémentation

Premier rendu



IV. Implémentation

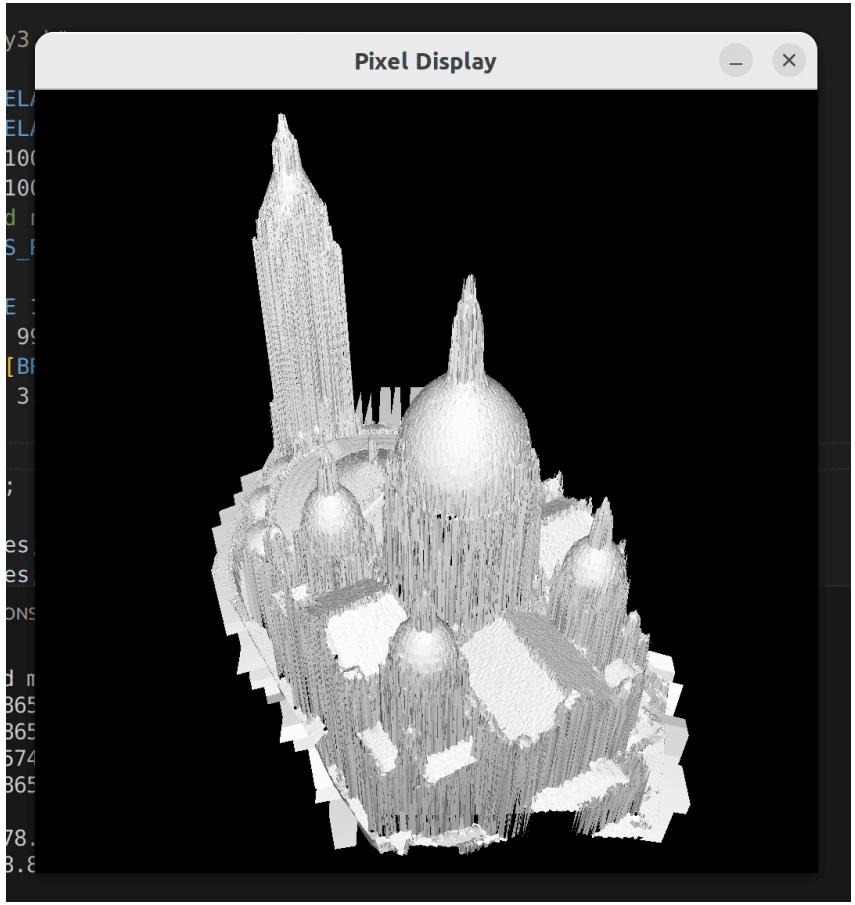
Extraction de données LiDAR



Logiciel CloudCompare

IV. Implémentation

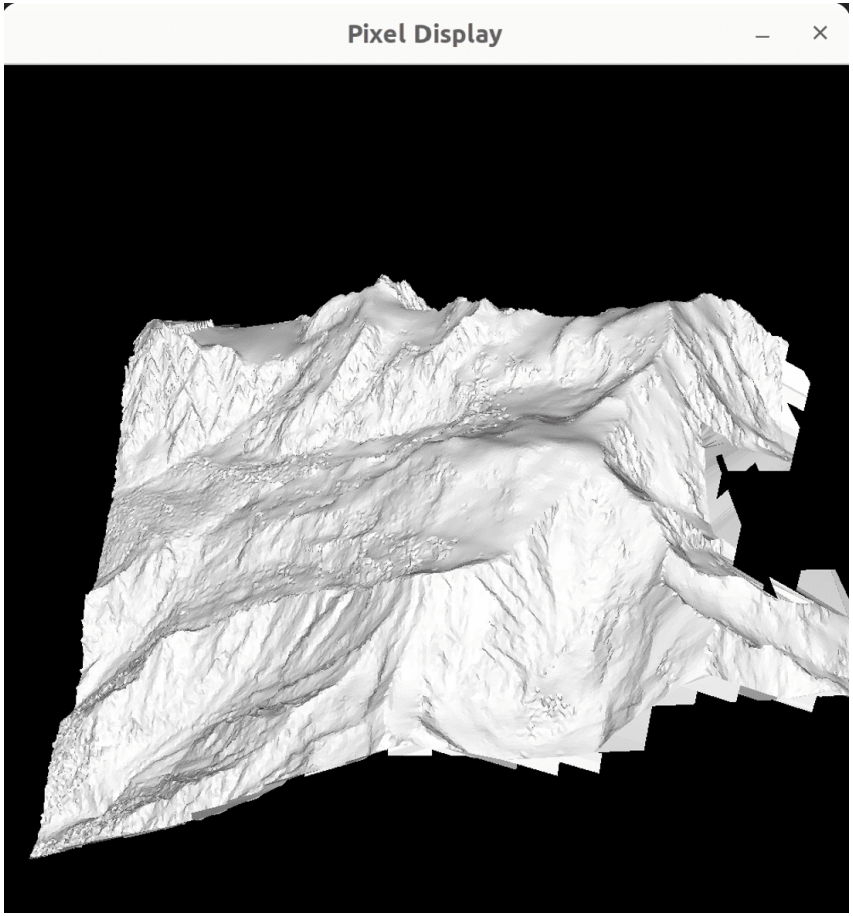
Rendu final

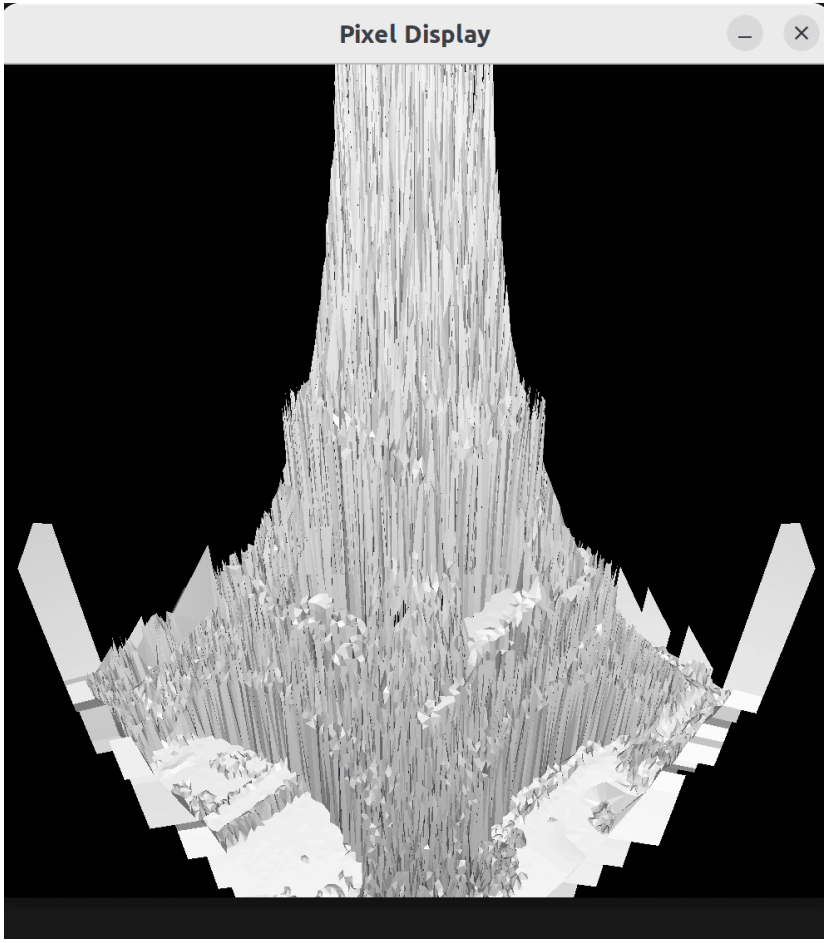



```
n_points : 100771
Triangulation terminée en 3.580163s
n_triangles : 201544
RENDU EN COURS --- 100%
Calculs d'intersection rayon/boîte : 9681046
Calculs d'intersection rayon/triangle : 1206841091
Rendu terminé en 344.726243s
Faire le rendu suivant ?
```

IV. Implémentation

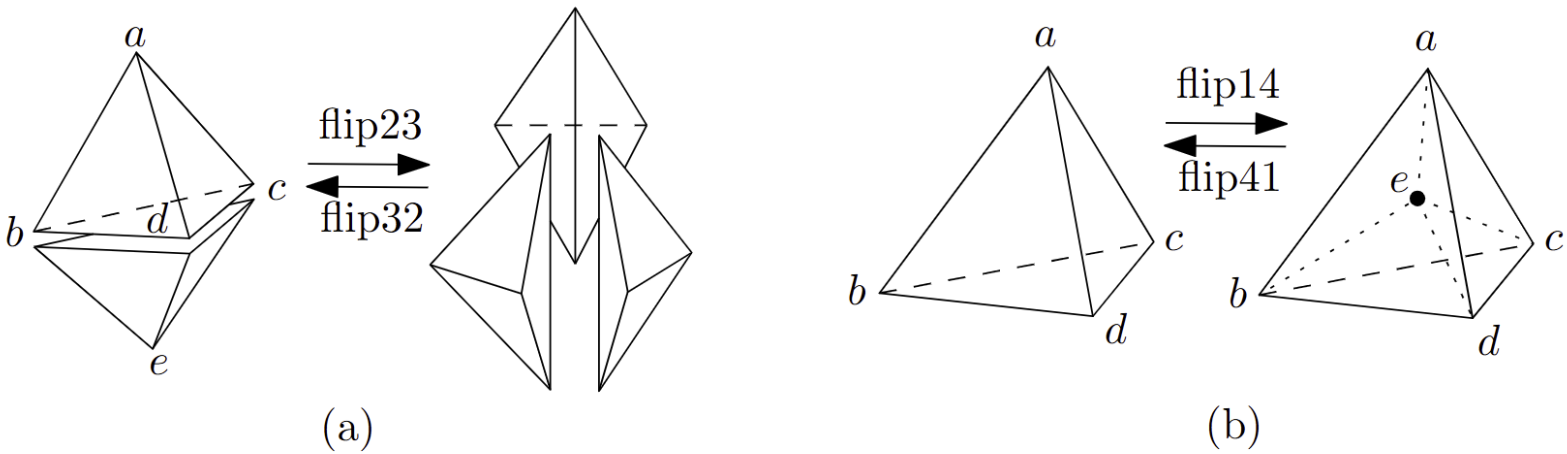
Autres rendus





V. Généralisation

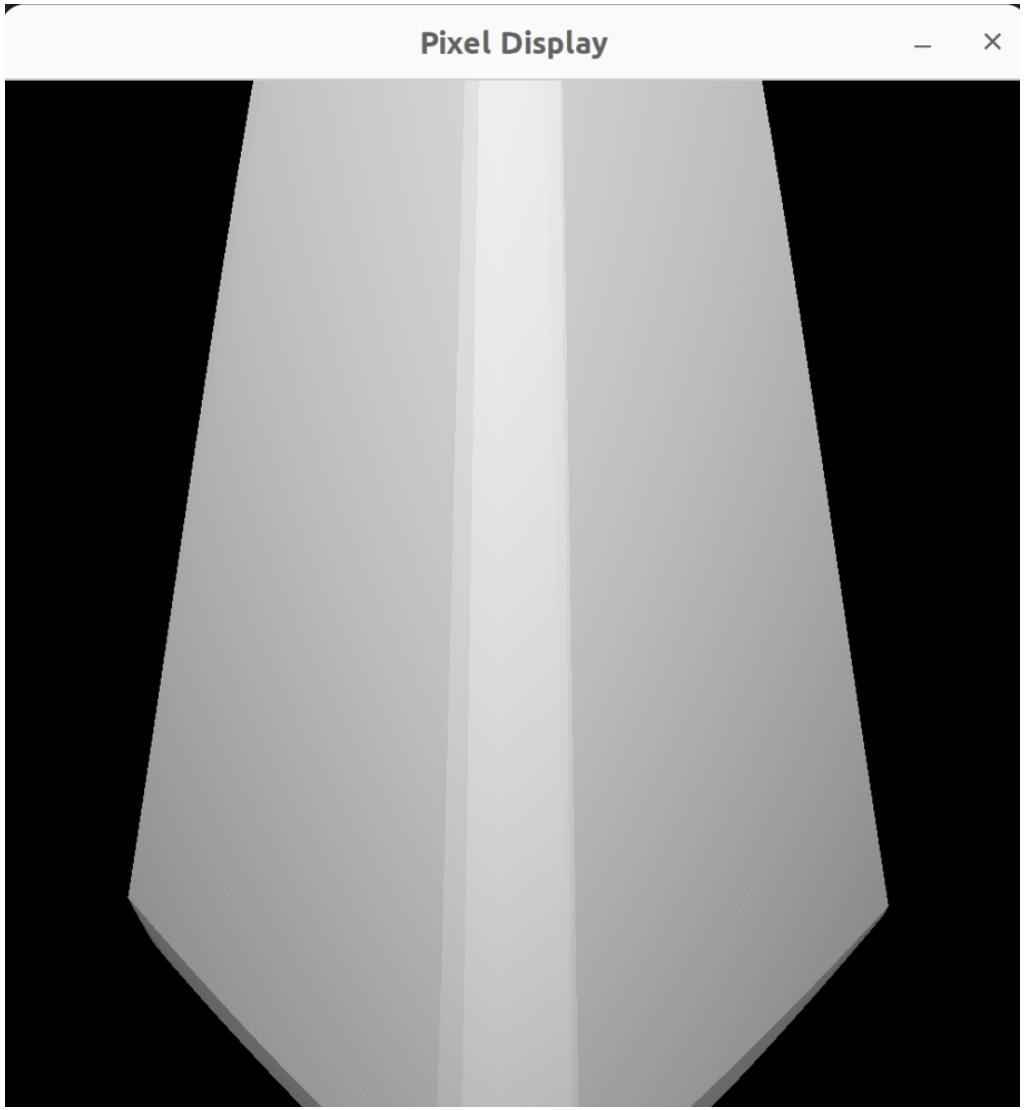
Bascules en 3D



Les opérateurs de bascule en 3D (Boguslawski et al.)

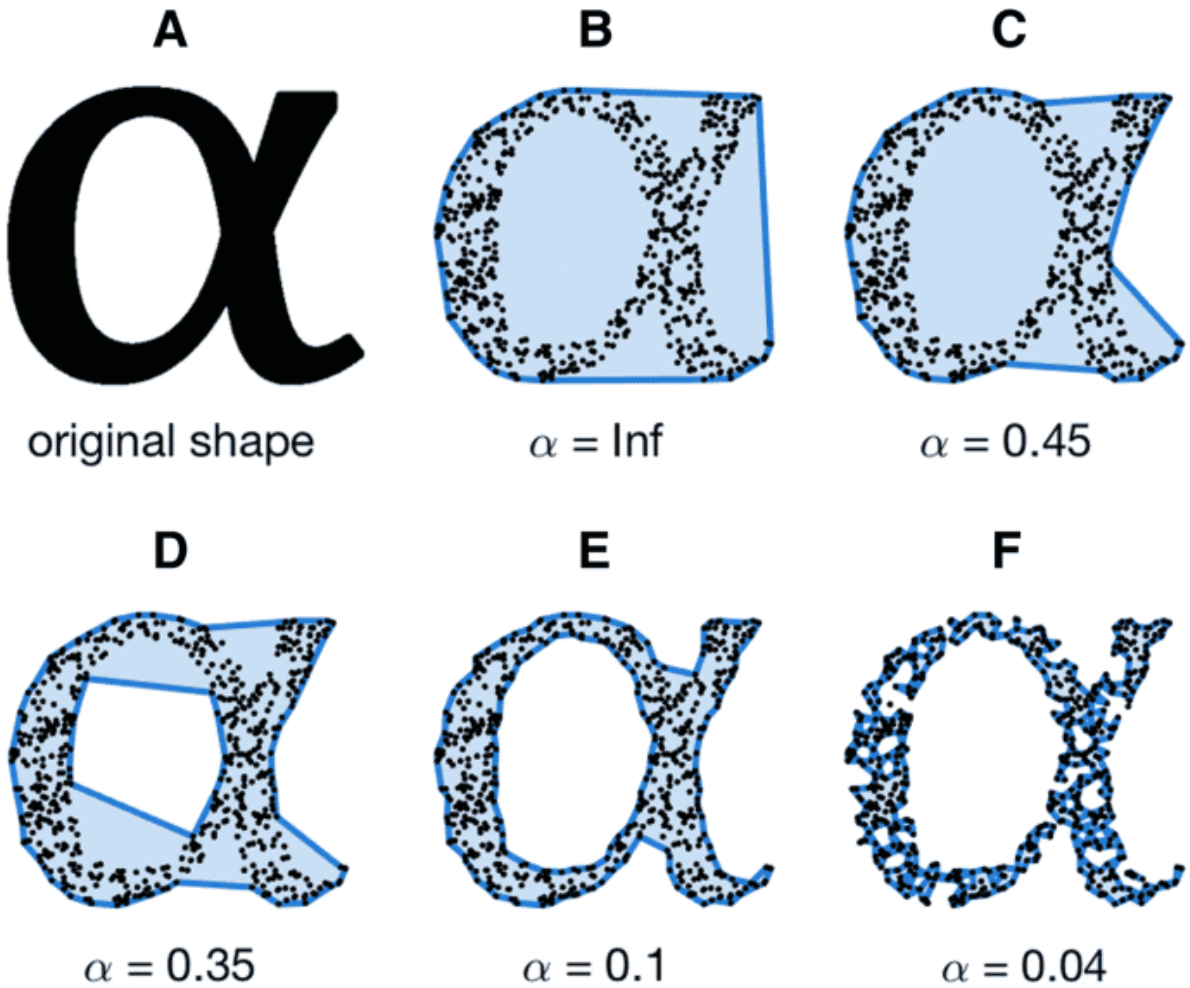
V. Généralisation

Perte de concavité



V. Généralisation

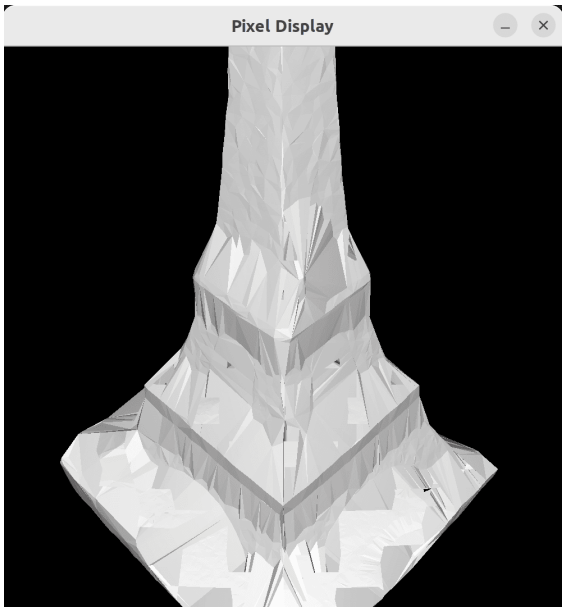
Alpha shapes



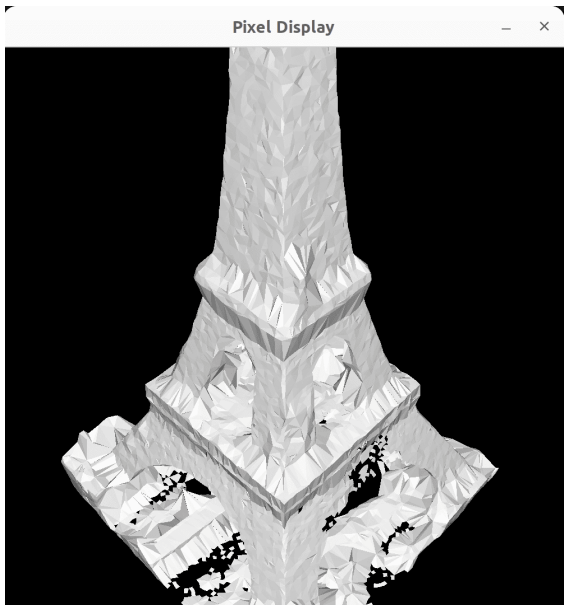
Alpha shapes (James D. Gardiner et al.)

V. Généralisation

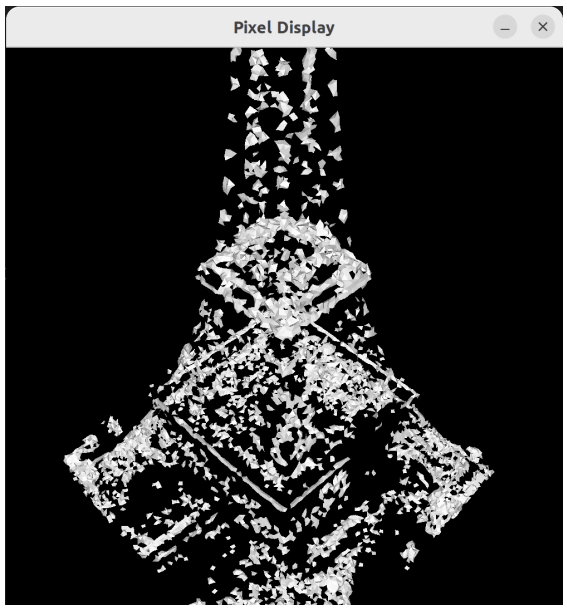
Rendu après filtrage



$\alpha = 10^9$



$\alpha = 10^7$



$\alpha = 10^5$

Merci pour votre attention.

Annexe

Initialiser les rayons

main.c

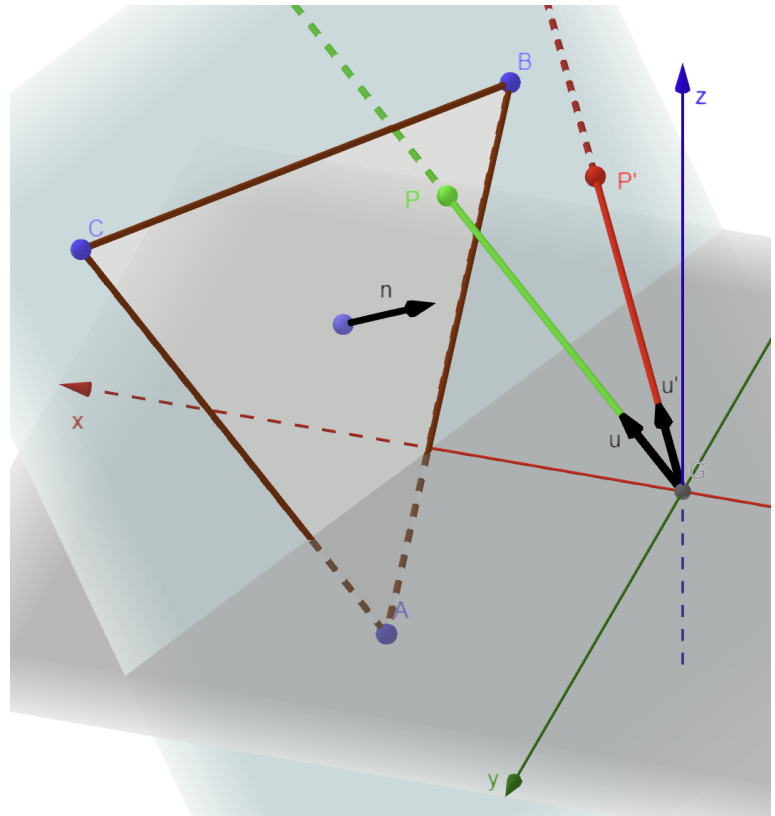
```
1   void initialise_camera(vector3** camera, double fov, double y_rot) {
2       for (int i = 0; i < SCREEN_I; i++) {
3           for (int j = 0; j < SCREEN_J; j++) {
4               double theta = (pi / 2) - (fov / 2)
5                                + fov * (i / (double)SCREEN_I);
6               double phi = (fov / 2) - fov * (j / (double)SCREEN_J);
7               vector3 v = {.x = 0.0, .y = 0.0, .z = 0.0};
8               v.x = sin(theta) * cos(phi);
9               v.y = sin(theta) * sin(phi);
10              v.z = cos(theta);
11              v = v_normalise(v);
12              v = v_rotate(v, 0, y_rot, 0);
13              camera[i][j] = v;
14          }
15      }
16  }
```

Annexe

Rendu d'un triangle

$$\vec{P} = \lambda \vec{u}$$

$$\begin{aligned} \vec{P} \in \Pi_{ABC} &\iff (\lambda \vec{u} - \vec{A}) \cdot \vec{n} = 0 \\ &\iff \lambda(\vec{u} \cdot \vec{n}) = \vec{A} \cdot \vec{n} \\ &\underset{(\vec{u}\cdot\vec{n}\neq 0)}{\iff} \lambda = \frac{\vec{A} \cdot \vec{n}}{\vec{u} \cdot \vec{n}} \end{aligned}$$



Annexe

Dualité, unicité

On considère $\mathcal{G}$ le graphe associé à $DT(\tilde{P})$.

Pour $p \in \tilde{P}$, $Vor(\tilde{P})[p] = \{x \in \mathbb{R}^2 : \forall q \in \tilde{P}, ||x - p|| \leq ||x - q||\}$.

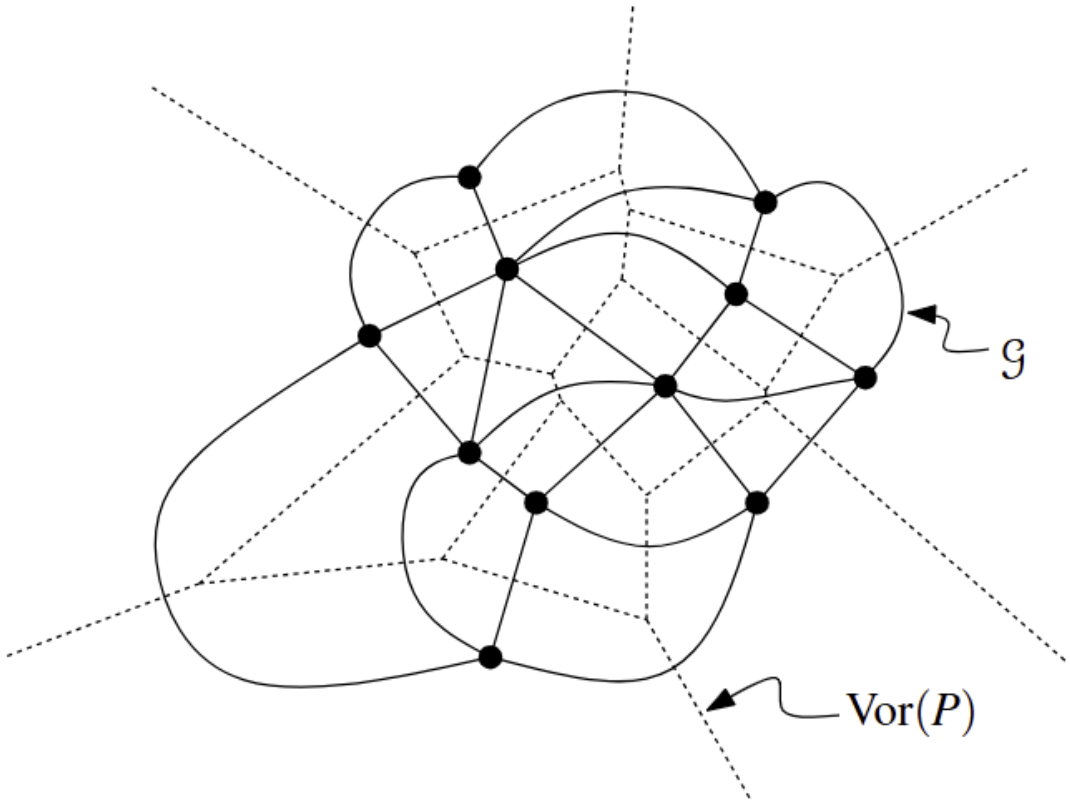
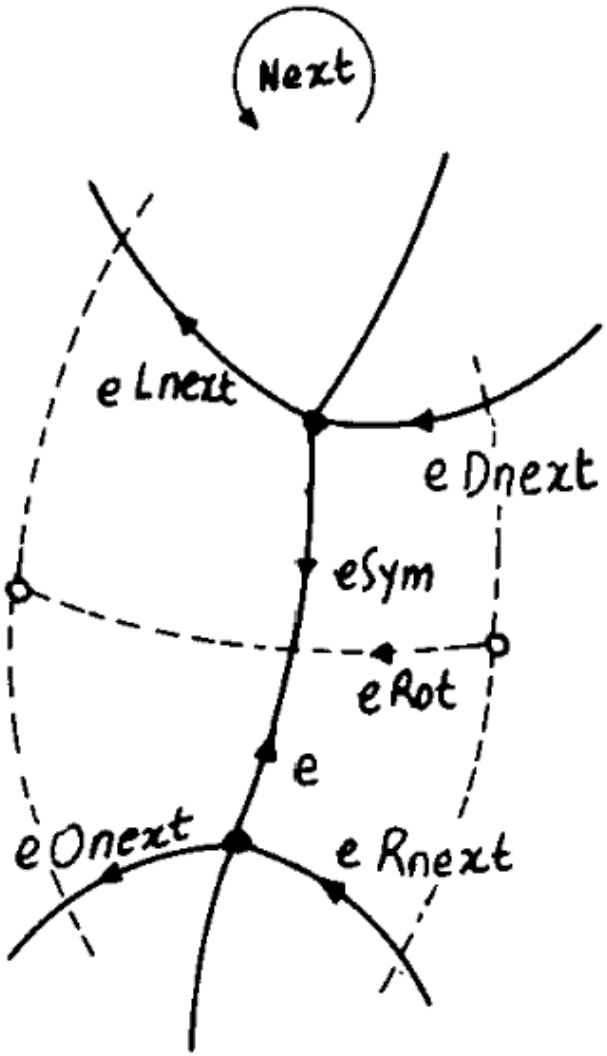


Diagramme de Voronoï (De Berg et al.)

Annexe

Navigation



Opérateurs de base (Guibas & Stolfi)

edge.c

```
1  int e_sym(edge* e) {
2    return e->rot->rot;
3  }
```

Annexe

Fonctions de navigation

edge.c

```
1   edge* e_sym(edge* e) {
2       return e->rot->rot;
3   }
4   edge* e_anti_rot(edge* e) {
5       return e->rot->rot->rot;
6   }
7   edge* e_anti_onext(edge* e) {
8       return e_rot(e_onext(e_rot(e)));
9   }
10  edge* e_dnext(edge* e) {
11      return e_sym(e_onext(e_sym(e)));
12  }
13  edge* e_anti_dnext(edge* e) {
14      return e_anti_rot(e_onext(e_anti_rot(e)));
15  }
16  edge* e_lnext(edge* e) {
17      return e_rot(e_onext(e_anti_rot(e)));
18  }
19  edge* e_rnext(edge* e) {
20      return e_anti_rot(e_onext(e_rot(e)));
21  }
```

Annexe

Fonction MakeEdge

edge.c

```
1   edge* e_make_edge() {
2       edge* e0 = (edge*)malloc(sizeof(edge));
3       edge* e1 = (edge*)malloc(sizeof(edge));
4       edge* e2 = (edge*)malloc(sizeof(edge));
5       edge* e3 = (edge*)malloc(sizeof(edge));
6       e0->onext = e0;
7       e1->onext = e3;
8       e2->onext = e2;
9       e3->onext = e1;
10      e0->rot = e1;
11      e1->rot = e2;
12      e2->rot = e3;
13      e3->rot = e0;
14      vector3 default_origin = v_create(0.0, 0.0, 0.0);
15      e0->origin = default_origin;
16      e1->origin = default_origin;
17      e2->origin = default_origin;
18      e3->origin = default_origin;
19      return e0;
20  }
```

Annexe

Fonctions de construction

edge.c

```
1   void e_splice(edge* a, edge* b) {
2       edge* alpha = e_rot(e_onext(a));
3       edge* beta = e_rot(e_onext(b));
4       edge* a_next = e_onext(b);
5       edge* b_next = e_onext(a);
6       edge* alpha_next = e_onext(beta);
7       edge* beta_next = e_onext(alpha);
8       a->onext = a_next;
9       b->onext = b_next;
10      alpha->onext = alpha_next;
11      beta->onext = beta_next;
12  }
13  edge* e_connect(edge* a, edge* b) {
14      edge* e = e_make_edge();
15      e->origin = e_sym(a)->origin;
16      e_sym(e)->origin = b->origin;
17      e_splice(e, e_lnext(a));
18      e_splice(e_sym(e), b);
19      return e;
20  }
```

edge.c

```
1   void e_remove(edge* e) {
2       e_splice(e, e_anti_onext(e));
3       e_splice(
4           e_sym(e),
5           e_anti_onext(e_sym(e))
6       );
7   }
8   void e_swap(edge* e) {
9       edge* a = e_anti_onext(e);
10      edge* b = e_anti_onext(e_sym(e));
11      e_splice(e, a);
12      e_splice(e_sym(e), b);
13      e_splice(e, e_lnext(a));
14      e_splice(e_sym(e), e_lnext(b));
15      e->origin = e_sym(a)->origin;
16      e_sym(e)->origin = e_sym(b)->origin;
17  }
```

Annexe

Fonction InCircle

delaunay2.c

```
1  bool d2_in_circle(vector3 a, vector3 b, vector3 c, vector3 p) {
2      return (a.x * a.x + a.y * a.y) * d2_triangle_area(b, c, p) -
3      (b.x * b.x + b.y * b.y) * d2_triangle_area(a, c, p) +
4      (c.x * c.x + c.y * c.y) * d2_triangle_area(a, b, p) -
5      (p.x * p.x + p.y * p.y) * d2_triangle_area(a, b, c) > 0;
6  }
```

Annexe

Fonctions auxiliaires

delaunay2.c

```
1   double d2_triangle_area(vector3 a, vector3 b, vector3 c) {
2       return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
3   }
4   bool d2_ccw(vector3 a, vector3 b, vector3 c) {
5       return d2_triangle_area(a, b, c) > 0;
6   }
7   bool d2_left_of(vector3 p, edge* e) {
8       return d2_ccw(p, e->origin, e_sym(e)->origin);
9   }
10  triangle d2_extract_dual_face(edge* e) {
11      edge* e1 = e_rot(e);
12      edge* e2 = e_rot(e_onext(e));
13      edge* e3 = e_rot(e_onext(e_onext(e)));
14      triangle t = t_create(e1->origin, e2->origin, e3->origin);
15      return t;
16  }
17  bool d2_seen_node(edge* e) {
18      edge* start = e;
19      if (e->data == -1) {
20          return true;
21      }
22      while (e_onext(e) != start) {
23          e = e_onext(e);
24          if (e->data == -1) {
25              return true;
26          }
27      }
28      return false;
29  }
```

Annexe

Fonction Locate

delaunay2.c

```
1   edge* d2_locate(vector3 p, edge* e0) {
2       edge* e = e0;
3       while (true) {
4           if (!d2_left_of(p, e)) {
5               e = e_sym(e);
6           }
7           else if (d2_left_of(p, e_onext(e))) {
8               e = e_onext(e);
9           }
10          else if (d2_left_of(p, e_anti_dnext(e))) {
11              e = e_anti_dnext(e);
12          }
13          else {
14              return e;
15          }
16      }
17  }
```

Annexe

Fonction Insert

delaunay2.c

```
1   void d2_insert(vector3 p, edge* e0) {
2       edge* e = d2_locate(p, e0);
3       edge* base = e_make_edge();
4       e_set_endpoints(base, e->origin, p);
5       e_splice(base, e);
6       edge* start = base;
7       while (e_onext(e) != start) {
8           base = e_connect(e, e_sym(base));
9           e = e_anti_onext(base);
10      }
11      while (e_onext(e) != start) {
12          edge* t = e_anti_onext(e);
13          if (!d2_left_of(e_sym(t)->origin, e) &&
14              d2_in_circle(e->origin, e_sym(t)->origin, e_sym(e)->origin, p)) {
15              e_swap(e);
16              e = e_anti_onext(e);
17          } else {
18              e = e_anti_lnext(e_onext(e));
19          }
20      }
21  }
```

Annexe

Fonction ExtractTriangles

delaunay2.c

```
1   int d2_extract_triangles(triangle* triangles, edge* e0) {
2       int n_triangles = 0;
3       edge** pile = (edge**)malloc(sizeof(edge*) * BIG_ARRAY);
4       int n_pile = 0;
5       pile[n_pile] = e0;
6       n_pile = n_pile + 1;
7       while (n_pile > 0 && n_pile < BIG_ARRAY) {
8           edge* e = pile[n_pile - 1];
9           n_pile = n_pile - 1;
10          if (e->data != -1) {
11              edge* next = e_sym(e);
12              edge* start = next;
13              while (e_onext(next) != start) {
14                  next = e_onext(next);
15                  pile[n_pile] = next;
16                  n_pile = n_pile + 1;
17              }
18          }
19          if (!d2_seen_node(e)) {
20              triangle t = d2_extract_dual_face(e);
21              triangles[n_triangles] = t;
22              n_triangles = n_triangles + 1;
23          }
24          e->data = -1;
25      }
26      return n_triangles;
27  }
```

Annexe

Démonstration du lemme

A, B et C vérifient

$$(x - x_O)^2 + (y - y_O)^2 = R^2$$

En développant,

$$(-2x_O)x + (-2y_O)y + (x^2 + y^2) + (x_O^2 + y_O^2 - R^2) = 0$$

Sous écriture matricielle

$$\underbrace{\begin{pmatrix} x_A & y_A & x_A^2 + y_A^2 & 1 \\ x_B & y_B & x_B^2 + y_B^2 & 1 \\ x_C & y_C & x_C^2 + y_C^2 & 1 \\ x_Q & y_Q & x_Q^2 + y_Q^2 & 1 \end{pmatrix}}_{M(Q)} \begin{pmatrix} -2x_O \\ -2y_O \\ 1 \\ x_O^2 + y_O^2 - R^2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ \lambda \end{pmatrix}$$

Annexe

Démonstration du lemme

- Si $Q \in \mathscr{C}$, $\lambda = 0$ et $M(Q)$ n'est pas inversible ; $\boxed{\det M(Q) = 0.}$
- Si $\det M(Q) = 0$, il existe $\alpha, \beta, \gamma, \delta$ non tous nuls tels que

$$\begin{pmatrix} x_A & y_A & x_A^2 + y_A^2 & 1 \\ x_B & y_B & x_B^2 + y_B^2 & 1 \\ x_C & y_C & x_C^2 + y_C^2 & 1 \\ x_Q & y_Q & x_Q^2 + y_Q^2 & 1 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \\ \gamma \\ \delta \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

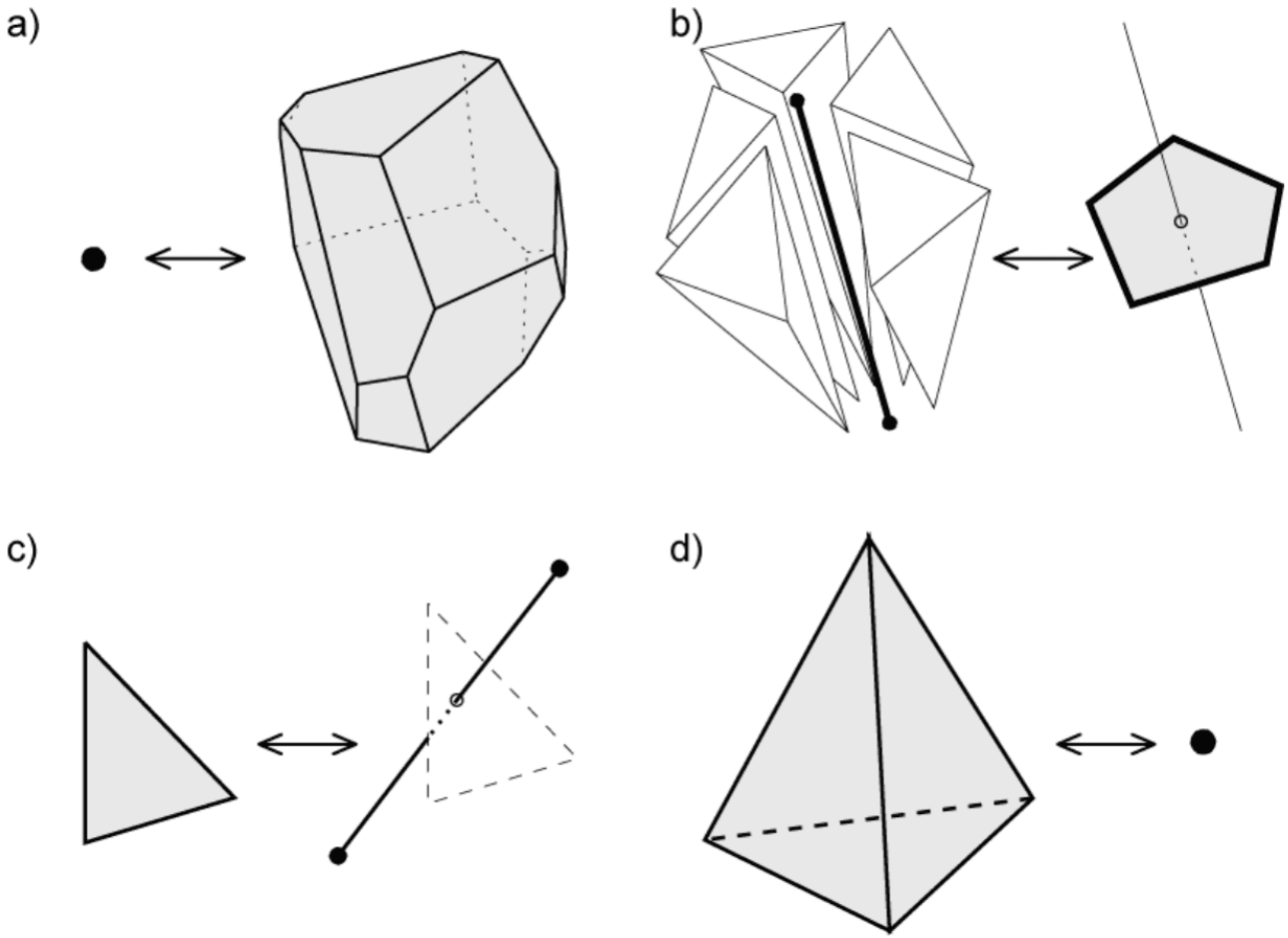
Donc A, B, C et Q vérifient

$$(x + \frac{\alpha}{2\gamma})^2 + (y + \frac{\beta}{2\gamma})^2 = \frac{\alpha^2 + \beta^2 - 4\gamma\delta}{4\gamma^2}$$

Donc ils appartiennent au même cercle ; $\boxed{Q \in \mathscr{C}.}$

Annexe

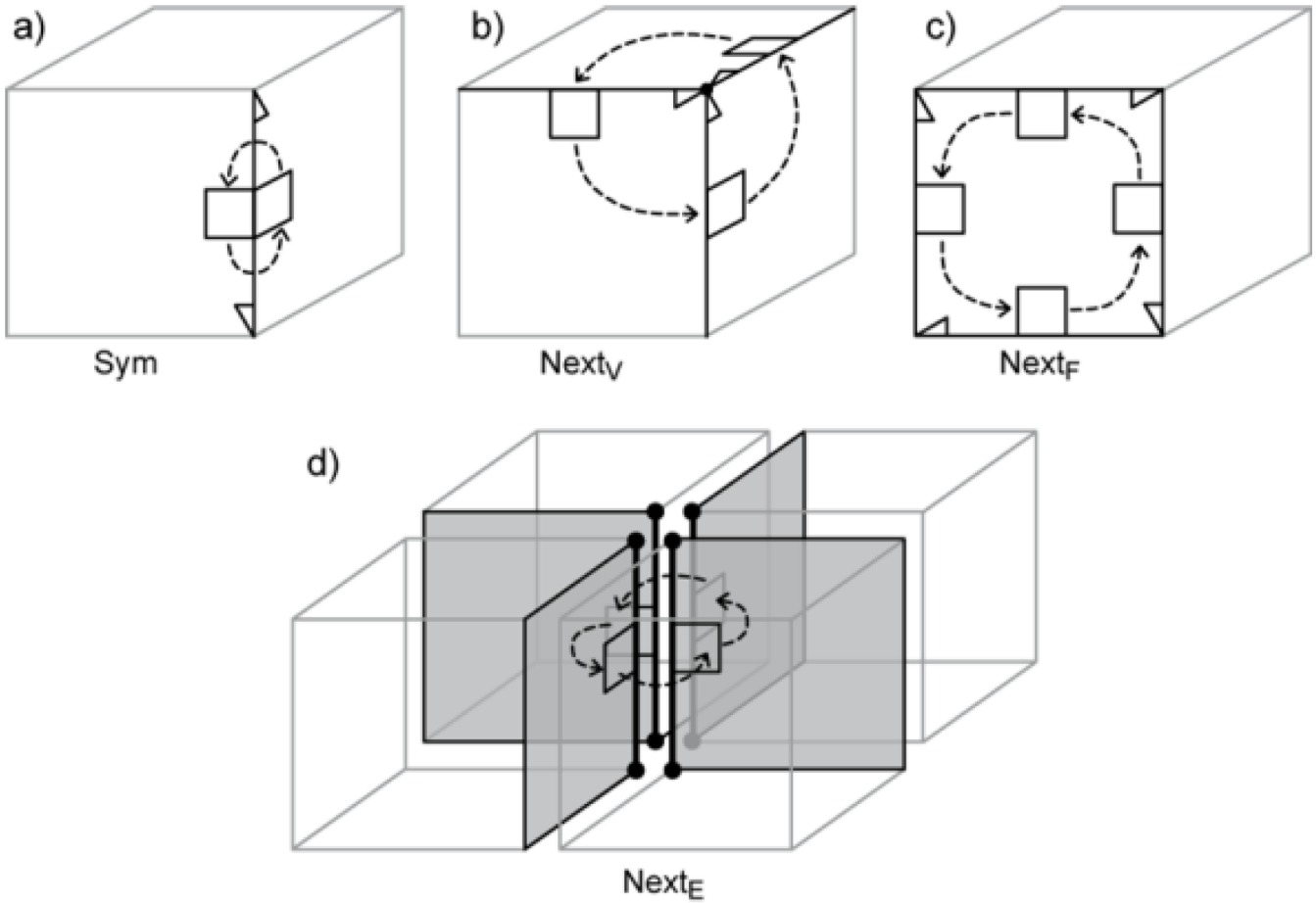
Dualité en trois dimensions



Lien primal / dual (Boguslawski et al.)

Annexe

Le dual half edge



```
1   #include <assert.h>
2   #include <stdbool.h>
3   #include <stddef.h>
4   #include <stdint.h>
5   #include <stdio.h>
6   #include <stdlib.h>
7   #include <time.h>
8   #include <math.h>
9   #include <unistd.h>
10  #include <SDL2/SDL.h>
11
12  #include "vector3.h"
13  #include "sphere.h"
14  #include "light.h"
15  #include "triangle.h"
16  #include "edge.h"
17  #include "delaunay2.h"
18  #include "colour.h"
19  #include "half.h"
20  #include "delaunay3.h"
21
22  #define SCREEN_J 1001 //Number of columns
23  #define SCREEN_I 1001 //Number of lines
24  #define BRIGHTNESS_RANGE 8
25  #define FRAMES 1
26  #define PIXEL_SIZE 1
27  #define BIG_ARRAY 3999999
28  const double pi = 3.141593;
29
30  //Prepare bounding boxes
31  const double x_min = 1400; //1700
32  const double y_min = -1900; //-1300
33  const double z_min = -5800; //-4700
34  const double x_max = 5200; //4900
35  const double y_max = 1900; //1300
36  const double z_max = -500; // 1900 (Sacre coeur(alpha2500))
```

```
37  const int box_x_amount = 4; //12
38  const int box_y_amount = 8; //22
39  const int box_z_amount = 8; //8
40
41  struct World {
42      int n_spheres;
43      int n_lights;
44      int n_triangles;
45      sphere* spheres;
46      light* lights;
47      triangle* triangles;
48  };
49  typedef struct World world;
50
51  colour** create_screen(void);
52  void delete_screen(double** screen);
53  vector3** create_camera(void);
54  void delete_camera(vector3** camera);
55  void initialise_camera(vector3** camera, double fov, double y_rot);
56  void render_accelerated(colour** screen, vector3** camera, world* world, int pixel_size);
57  int extract_points(FILE* fh, vector3* points);
58
59  bool box_intersects_vector(vector3 box_center, double x_r, double y_r, double z_r, vector3 v);
60  bool box_intersects_triangle(vector3 box_center, double x_r, double y_r, double z_r, triangle t);
61  void tetgen_write_points(vector3* points, int n_points);
62  int tetgen_read_triangles(triangle* triangles, vector3* points);
63  int tetgen_alpha_filter(triangle* triangles, double alpha_min, double alpha_max, vector3* points);
64  double tetrahedron_circumsphere_volume(vector3 a, vector3 b, vector3 c, vector3 d);
65
66  int main(void) {
67
68      //Bounding triangle access
69      edge* e0 = d2_bounding_triangle();
70
71      //Read file points
72      FILE* fh = fopen("data/mont_blanc_50k.txt", "r");
```

```
73      vector3* points = (vector3*)malloc(sizeof(vector3) * BIG_ARRAY);
74      int n_points = extract_points(fh, points);
75      printf("n_points : %i\n", n_points);
76      //2D DT
77      for (int i = 0; i < n_points; i++) {
78          vector3 p = points[i];
79          d2_insert(p, e0);
80      }
81      fclose(fh);
82
83      //Define screen
84      colour** screen = create_screen();
85
86      //Define camera
87      vector3** camera = create_camera();
88      initialise_camera(camera, pi / 4, -pi / 4);
89
90      //Define light objects
91      light light_1 = l_create(v_create(0, 0, 50), 1000);
92
93      //Define the world
94      const int n_spheres = 0;
95      const int n_lights = 1;
96      sphere* spheres = (sphere*)malloc(sizeof(sphere) * n_spheres);
97      light* lights = (light*)malloc(sizeof(light) * n_lights);
98
99      //Fill the world
100     triangle* triangles = (triangle*)malloc(sizeof(triangle) * BIG_ARRAY);
101
102     const int n_triangles = d2_extract_triangles(triangles, e_rot(e0));
103     printf("n_triangles: %i\n", n_triangles);
104
105
106     printf("Faire le rendu ? ");
107     fflush(stdout);
108     char answer_render;
```

```
109         scanf(" %c", &answer_render);
110         if (answer_render != 'y') return 0;
111 
112         lights[0] = light_1;
113 
114         world world_1 = {
115             .n_spheres = n_spheres,
116             .n_lights = n_lights,
117             .n_triangles = n_triangles,
118             .spheres = spheres,
119             .lights = lights,
120             .triangles = triangles
121         };
122 
123         //SDL business
124         SDL_Window *window = SDL_CreateWindow(
125             "Pixel Display",
126             SDL_WINDOWPOS_CENTERED,
127             SDL_WINDOWPOS_CENTERED,
128             SCREEN_J, SCREEN_I, 0
129         );
130         SDL_Renderer *renderer = SDL_CreateRenderer(window, -1, SDL_RENDERER_ACCELERATED);
131         SDL_SetRenderDrawColor(renderer, 0, 0, 0, 255); //Black background
132         SDL_RenderClear(renderer);
133 
134         //Frame loop
135         for (int frame = 0; frame < FRAMES; frame++) {
136             //Render world into screen
137             clock_t begin = clock();
138             render_accelerated(screen, camera, &world_1, PIXEL_SIZE + FRAMES - frame - 1);
139             clock_t end = clock();
140             //Display screen
141             for (int i = 0; i < SCREEN_I; i++) { //Iterate over row (vertically)
142                 for (int j = 0; j < SCREEN_J; j++) { //Iterate over column (horizontally)
143                     colour c = screen[i][j];
144                     SDL_SetRenderDrawColor(renderer,
```

```
145                     (int) c.r * c.brightness,
146                     (int) c.g * c.brightness,
147                     (int) c.b * c.brightness,
148                     255
149                 );
150                 SDL_RenderDrawPoint(renderer, j, i);
151             }
152         }
153         usleep(REFRESH_DELAY);
154         //SDL business
155         SDL_RenderPresent(renderer);
156         //Time spent
157         double time_spent = (double)(end - begin) / CLOCKS_PER_SEC;
158         printf("Rendu terminé en %lfs\n", time_spent);
159         //Ask for next frame
160         printf("Faire le rendu suivant ? ");
161         fflush(stdout);
162         char answer;
163         scanf(" %c", &answer);
164         if (answer != 'y') break;
165     }
166 
167     //SDL business
168     SDL_Delay(DISPLAY_DELAY);
169     SDL_DestroyRenderer(renderer);
170     SDL_DestroyWindow(window);
171     SDL_Quit();
172 
173     //Delete objects
174     delete_camera(camera);
175     free(spheres);
176     free(lights);
177     free(triangles);
178 
179     return 0;
180 }
```

```
181
182 colour** create_screen(void) {
183     colour** screen = (colour**)malloc(sizeof(colour*) * SCREEN_J); //2D array that holds lines
184     for (int i = 0; i < SCREEN_I; i++) {
185         colour* line = (colour*)malloc(sizeof(colour) * SCREEN_I); //1D array representing a line
186         screen[i] = line;
187     }
188     return screen;
189 }
190
191 void delete_screen(double** screen) {
192     for (int i = 0; i < SCREEN_I; i++) {
193         free(screen[i]); //Delete line
194     }
195     free(screen);
196 }
197
198 vector3** create_camera(void) {
199     vector3** camera = (vector3**)malloc(sizeof(vector3*)*SCREEN_J); //2D array that holds lines
200     for (int i = 0; i < SCREEN_I; i++) {
201         vector3* line = (vector3*)malloc(sizeof(vector3)*SCREEN_I); //1D array representing a line
202         camera[i] = line;
203     }
204     return camera;
205 }
206
207 void delete_camera(vector3** camera) {
208     for (int i = 0; i < SCREEN_I; i++) {
209         free(camera[i]); //Delete line
210     }
211     free(camera);
212 }
213
214 void initialise_camera(vector3** camera, double fov, double y_rot) {
215     //fov is the angle range in radians
216     for (int i = 0; i < SCREEN_I; i++) {
```

```
217             for (int j = 0; j < SCREEN_J; j++) {
218                 double theta = (pi / 2) - (fov / 2) + fov * (i / (double)SCREEN_I);
219                 double phi = (fov / 2) - fov * (j / (double)SCREEN_J);
220                 //Initialise vectors correctly
221                 vector3 v = {.x = 0.0, .y = 0.0, .z = 0.0};
222                 v.x = sin(theta) * cos(phi);
223                 v.y = sin(theta) * sin(phi);
224                 v.z = cos(theta);
225                 v = v_normalise(v);
226                 v = v_rotate(v, 0, y_rot, 0);
227                 camera[i][j] = v;
228             }
229         }
230     }
231
232     void render_accelerated(colour** screen, vector3** camera, world* world, int pixel_size) {
233
234         //Prepare bounding boxes
235         int box_amount = box_x_amount * box_y_amount * box_z_amount;
236         double box_x_radius = (int)(x_max - x_min) / (box_x_amount * 2);
237         double box_y_radius = (int)(y_max - y_min) / (box_y_amount * 2);
238         double box_z_radius = (int)(z_max - z_min) / (box_z_amount * 2);
239         double box_x_origin = x_min;
240         double box_y_origin = y_min;
241         double box_z_origin = z_min;
242         vector3* boxes = (vector3*)malloc(sizeof(vector3) * box_amount);
243         int c = 0;
244         for (int i = 0; i < box_x_amount; i++) {
245             for (int j = 0; j < box_y_amount; j++) {
246                 for (int k = 0; k < box_z_amount; k++) {
247                     vector3 box_center = v_create(
248                         box_x_origin + box_x_radius + 2 * box_x_radius * i,
249                         box_y_origin + box_y_radius + 2 * box_y_radius * j,
250                         box_z_origin + box_z_radius + 2 * box_z_radius * k
251                     );
252                     boxes[c] = box_center;
```

```
253                 c = c + 1;
254             }
255         }
256     }
257
258     //Fill bounding boxes
259     triangle** boxes_triangles = (triangle**)malloc(sizeof(triangle*) * box_amount);
260     int* boxes_size = (int*)malloc(sizeof(int) * box_amount);
261     for (int l = 0; l < box_amount; l++) {
262         vector3 box_center = boxes[l];
263         triangle* box_triangles = (triangle*)malloc(sizeof(triangle) * BIG_ARRAY); //TODO this in unre
264         int box_size = 0;
265         for (int k = 0; k < world->n_triangles; k++) {
266             triangle t = world->triangles[k];
267             if (box_intersects_triangle(box_center, box_x_radius, box_y_radius, box_z_radius, t)) {
268                 box_triangles[box_size] = t;
269                 box_size = box_size + 1;
270             }
271         }
272         boxes_size[l] = box_size;
273         boxes_triangles[l] = box_triangles;
274     }
275
276     int done = 0;
277     int box_intersections = 0; //STATS
278     int triangle_tests = 0; //STATS
279     for (int i = 0; i < SCREEN_I / pixel_size; i++) { //Iterate over row (vertically)
280
281         //For the RENDERING text
282         int done_new = (100 * i) / (SCREEN_I / pixel_size) + 2;
283         if (done_new > done) {
284             printf("\rRENDU EN COURS --- %i%%", done);
285             fflush(stdout);
286             done = done_new;
287         }
288
```

```
289             for (int j = 0; j < SCREEN_J / pixel_size; j++) { //Iterate over column (horizontally)
290                 vector3 v = camera[i * pixel_size + pixel_size / 2][j * pixel_size + pixel_size / 2];
291 
292                 //Spheres
293                 double s_min_distance = 9999.0;
294                 bool s_intersection_exists = false;
295                 sphere s_min;
296                 for (int k = 0; k < world->n_spheres; k++) {
297                     sphere s = world->spheres[k];
298                     bool intersects = s_intersects(s, v);
299                     if (intersects) {
300                         //Calculate distance between source and point of intersection
301                         double intersects_distance = s_intersection_distance(s, v);
302                         if (intersects_distance < s_min_distance) {
303                             s_min_distance = intersects_distance;
304                             s_intersection_exists = true;
305                             s_min = s;
306                         }
307                     }
308                 }
309 
310                 //Triangles
311                 double t_min_distance = 9999.0;
312                 bool t_intersection_exists = false;
313                 triangle t_min;
314 
315                 //Accelerated rendering
316                 for (int l = 0; l < box_amount; l++) {
317                     //Does the ray intersect this box? Using ray-AABB intersection
318                     vector3 box_center = boxes[l];
319 
320                     if (box_intersects_vector(box_center, box_x_radius, box_y_radius, box_z_radius, v)) {
321                         //It does, check all triangles inside
322                         int box_size = boxes_size[l];
323                         box_intersections = box_intersections + 1; //STATS
324                         triangle_tests = triangle_tests + box_size; //STATS
```

```
325                     for (int k = 0; k < box_size; k++) {
326                         triangle t = boxes_triangles[l][k];
327                         bool intersects = t_intersects(t, v);
328                         if (intersects) {
329                             //Calculate distance between source and point of intersection
330                             double intersects_distance = t_intersection_distance(t, v);
331                             if (intersects_distance < t_min_distance) {
332                                 t_min_distance = intersects_distance;
333                                 t_intersection_exists = true;
334                                 t_min = t;
335                             }
336                         }
337                     }
338                 }
339             }
340 
341             //Determine pixel to print
342             double brightness;
343             colour c;
344             if (!s_intersection_exists && !t_intersection_exists) { //No intersection
345                 brightness = 0;
346             }
347             else if (s_min_distance < t_min_distance) { //Sphere intersection
348                 //Calculate distance between light source and point of intersection
349                 double intersects_distance = s_intersection_distance(s_min, v);
350                 double light_distance = v_norm(v_sub(world->lights[0].position, v_mul(v, intersects_di
351                 //Calculate normal vector
352                 vector3 normal = s_intersection_normal(s_min, v);
353                 //Calculate other useful vectors
354                 vector3 intersection = v_mul(v, intersects_distance);
355                 vector3 light_to_intersection = v_sub(intersection, world->lights[0].position);
356                 //Apply shaders to get brightness
357                 if (v_dot(normal, light_to_intersection) > 0) {
358                     //Intersection point is not illuminated
359                     brightness = 0;
360                 }
```

```
361                 else {
362                     brightness = v_dot(normal, v_normalise(light_to_intersection));
363                     brightness = fabs(brightness);
364                     brightness = fmin(1, brightness);
365                 }
366             }
367             else { //Triangle intersection
368                 //Calculate distance between light source and point of intersection
369                 double intersects_distance = t_intersection_distance(t_min, v);
370                 double light_distance = v_norm(v_sub(world->lights[0].position, v_mul(v, intersects_di
371                 //Calculate normal vector
372                 vector3 normal = t_intersection_normal(t_min, v);
373                 //Calculate other useful vectors
374                 vector3 intersection = v_mul(v, intersects_distance);
375                 vector3 light_to_intersection = v_sub(intersection, world->lights[0].position);
376                 //Apply shaders to get brightness
377                 if (v_dot(normal, light_to_intersection) > 0) {
378                     //Intersection point is not illuminated
379                     brightness = 0;
380                 }
381                 else {
382                     brightness = v_dot(normal, v_normalise(light_to_intersection));
383                     brightness = fabs(brightness);
384                     brightness = sqrt(brightness);
385                     brightness = fmin(1, brightness);
386                 }
387                 c = t_min.c;
388             }
389 
390             //Set pixel
391             colour pixel = c;
392             pixel.brightness = brightness;
393             for (int k = 0; k < pixel_size; k++) {
394                 for (int l = 0; l < pixel_size; l++) {
395                     int ii = pixel_size * i + k;
396                     int jj = pixel_size * j + l;
```

```
397                     if (ii < SCREEN_I && jj < SCREEN_J) {
398                         screen[ii][jj] = pixel;
399                     }
400                 }
401             }
402         }
403     }
404     printf("\n"); //For the RENDERING text
405     printf("Calculs d'intersection rayon/boîte : %i\n", box_intersections);
406     printf("Calculs d'intersection rayon/triangle : %i\n", triangle_tests);
407 }
408
409 int extract_points(FILE* fh, vector3* points) {
410     int n_points = 0;
411     double x;
412     double y;
413     double z;
414     while (fscanf(fh, "%lf %lf %lf%*[^\n]", &x, &y, &z) == 3) {
415         //Fill the table
416         vector3 v = v_create(x, y, z);
417         points[n_points] = v;
418         n_points = n_points + 1;
419     }
420
421     vector3 v_mini = v_min(points, n_points);
422     vector3 v_maxi = v_max(points, n_points);
423     printf("min and max and delta and moy:\n");
424     v_print(v_mini);
425     v_print(v_maxi);
426     vector3 v_delta = v_sub(v_maxi, v_mini);
427     vector3 v_moy = v_mul(v_add(v_mini, v_maxi), 0.5);
428     v_print(v_delta);
429     v_print(v_moy);
430
431     vector3 v_offset = v_create(3300, 0, -2500);
432     double expand = 0.2;
```

```
433         for (int i = 0; i < n_points; i++) {
434             vector3 v = points[i];
435             vector3 u = v_add(v_mul(v_sub(v, v_moy), expand), v_offset);
436             points[i] = u;
437         }
438 
439         vector3 v_mini_2 = v_min(points, n_points);
440         vector3 v_maxi_2 = v_max(points, n_points);
441         printf("min and max:\n");
442         v_print(v_mini_2);
443         v_print(v_maxi_2);
444 
445         return n_points;
446     }
447 
448     bool box_intersects_vector(vector3 box_center, double x_r, double y_r, double z_r, vector3 v) {
449         double qx_1 = (box_center.x - x_r) / v.x;
450         double qx_2 = (box_center.x + x_r) / v.x;
451         double qx_min = fmin(qx_1, qx_2);
452         double qx_max = fmax(qx_1, qx_2);
453         double qy_1 = (box_center.y - y_r) / v.y;
454         double qy_2 = (box_center.y + y_r) / v.y;
455         double qy_min = fmin(qy_1, qy_2);
456         double qy_max = fmax(qy_1, qy_2);
457         double qz_1 = (box_center.z - z_r) / v.z;
458         double qz_2 = (box_center.z + z_r) / v.z;
459         double qz_min = fmin(qz_1, qz_2);
460         double qz_max = fmax(qz_1, qz_2);
461         double q_min = fmax(fmax(qx_min, qy_min), qz_min);
462         double q_max = fmin(fmin(qx_max, qy_max), qz_max);
463         return q_min <= q_max;
464     }
465 
466     bool box_intersects_triangle(vector3 box_center, double x_r, double y_r, double z_r, triangle t) {
467         return
468             (abs(t.point_1.x - box_center.x) < x_r &&
```

```
469             abs(t.point_1.y - box_center.y) < y_r &&
470             abs(t.point_1.z - box_center.z) < z_r)
471             ||
472             (abs(t.point_2.x - box_center.x) < x_r &&
473             abs(t.point_2.y - box_center.y) < y_r &&
474             abs(t.point_2.z - box_center.z) < z_r)
475             ||
476             (abs(t.point_3.x - box_center.x) < x_r &&
477             abs(t.point_3.y - box_center.y) < y_r &&
478             abs(t.point_3.z - box_center.z) < z_r);
479     }
480
481     void tetgen_write_points(vector3* points, int n_points) {
482         FILE* fh = fopen("tetgen/points.node", "w");
483         fprintf(fh, "%d 3 0 0\n", n_points);
484         for (int i = 0; i < n_points; i++) {
485             fprintf(fh, "%d %.17g %.17g %.17g\n", i, points[i].x, points[i].y, points[i].z);
486         }
487         fclose(fh);
488     }
489
490
491     int tetgen_read_triangles(triangle* triangles, vector3* points) {
492         FILE* fh = fopen("tetgen/points.1.face", "r");
493         int n_triangles;
494         int has_boundary;
495         fscanf(fh, "%d %d", &n_triangles, &has_boundary);
496         for (int i = 0; i < n_triangles; i++) {
497             int n_triangle;
498             int k1;
499             int k2;
500             int k3;
501             if (has_boundary) {
502                 int marker;
503                 fscanf(fh, "%d %d %d %d %d", &n_triangle, &k1, &k2, &k3, &marker);
504             } else {
```

```
505                 fscanf(fh, "%d %d %d %d", &n_triangle, &k1, &k2, &k3);
506             }
507             triangles[i] = t_create(points[k1], points[k2], points[k3]);
508         }
509         fclose(fh);
510         return n_triangles;
511     }
512
513     struct Ele {
514         int k1;
515         int k2;
516         int k3;
517         int k4;
518         int n1;
519         int n2;
520         int n3;
521         int n4;
522         bool keep;
523     };
524
525     int tetgen_alpha_filter(triangle* triangles, double alpha_min, double alpha_max, vector3* points) {
526
527         //Read and initialise tets, set points and keep
528         struct Ele* tets = (struct Ele*)malloc(sizeof(struct Ele) * BIG_ARRAY);
529         FILE* f1 = fopen("tetgen/points.1.ele", "r");
530         int n_tets;
531         int bin;
532         fscanf(f1, "%d %d %d", &n_tets, &bin, &bin);
533         for (int i = 0; i < n_tets; i++) {
534             int n_tet;
535             int k1;
536             int k2;
537             int k3;
538             int k4;
539             fscanf(f1, "%d %d %d %d %d", &n_tet, &k1, &k2, &k3, &k4);
540             struct Ele tet = (struct Ele){
```

```
541                 .k1 = k1, .k2 = k2, .k3 = k3, .k4 = k4,
542                 .n1 = 0, .n2 = 0, .n3 = 0, .n4 = 0,
543                 .keep = false
544             };
545             double volume = tetrahedron_circumsphere_volume(points[k1], points[k2], points[k3], points[k4]
546             if (fabs(volume) > alpha_min && fabs(volume) < alpha_max) {
547                 tet.keep = true;
548             }
549             tets[i] = tet;
550         }
551         fclose(f1);
552 
553         //Read all neighbours, set neighbours
554         FILE* f2 = fopen("tetgen/points.1.neigh", "r");
555         fscanf(f2, "%d %d", &n_tets, &bin);
556         for (int i = 0; i < n_tets; i++) {
557             int n_tet;
558             int n1;
559             int n2;
560             int n3;
561             int n4;
562             fscanf(f1, "%d %d %d %d %d", &n_tet, &n1, &n2, &n3, &n4);
563             tets[i].n1 = n1;
564             tets[i].n2 = n2;
565             tets[i].n3 = n3;
566             tets[i].n4 = n4;
567         }
568         fclose(f2);
569 
570         //Extract triangles of ext
571         int n_triangles = 0;
572         for (int i = 0; i < n_tets; i++) {
573             struct Ele tet = tets[i];
574             if (tet.keep) {
575                 vector3 a = points[tet.k1];
576                 vector3 b = points[tet.k2];
```

```
577                 vector3 c = points[tet.k3];
578                 vector3 d = points[tet.k4];
579                 triangle t1 = t_create(a, b, c);
580                 triangle t2 = t_create(a, b, d);
581                 triangle t3 = t_create(c, a, d);
582                 triangle t4 = t_create(b, c, d);
583                 if (!tets[tet.n4].keep) {
584                     triangles[n_triangles] = t1;
585                     n_triangles = n_triangles + 1;
586                 }
587                 if (!tets[tet.n3].keep) {
588                     triangles[n_triangles] = t2;
589                     n_triangles = n_triangles + 1;
590                 }
591                 if (!tets[tet.n2].keep) {
592                     triangles[n_triangles] = t3;
593                     n_triangles = n_triangles + 1;
594                 }
595                 if (!tets[tet.n1].keep) {
596                     triangles[n_triangles] = t4;
597                     n_triangles = n_triangles + 1;
598                 }
599             }
600         }
601         return n_triangles;
602     }
603
604     double tetrahedron_circumsphere_volume(vector3 a, vector3 b, vector3 c, vector3 d) {
605         vector3 u = {b.x - a.x, b.y - a.y, b.z - a.z};
606         vector3 v = {c.x - a.x, c.y - a.y, c.z - a.z};
607         vector3 w = {d.x - a.x, d.y - a.y, d.z - a.z};
608         double uu = v_dot(u, u);
609         double vv = v_dot(v, v);
610         double ww = v_dot(w, w);
611         vector3 cross_vw = v_cross(v, w);
612         vector3 cross_wu = v_cross(w, u);
```

```
613         vector3 cross_uv = v_cross(u, v);
614         double denom = 2.0 * v_dot(u, cross_vw);
615         vector3 numerator;
616         numerator.x = uu * cross_vw.x + vv * cross_wu.x + ww * cross_uv.x;
617         numerator.y = uu * cross_vw.y + vv * cross_wu.y + ww * cross_uv.y;
618         numerator.z = uu * cross_vw.z + vv * cross_wu.z + ww * cross_uv.z;
619         vector3 o;
620         o.x = a.x + numerator.x / denom;
621         o.y = a.y + numerator.y / denom;
622         o.z = a.z + numerator.z / denom;
623         double r = v_norm(v_sub(o, a));
624         double volume = (4.0 / 3.0) * pi * r * r * r;
625         return volume;
626     }
```

```c
#include <assert.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

#include "triangle.h"
#include "vector3.h"
#include "colour.h"

triangle t_create(vector3 point_1, vector3 point_2, vector3 point_3) {
    double avg_z = (point_1.z + point_2.z + point_3.z) / 3.0;
    colour c = c_create_white();
    triangle t = {.point_1 = point_1, .point_2 = point_2, .point_3 = point_3, .c = c};
    return t;
}

bool t_intersects(triangle t, vector3 v) {
    //Get n, direction of the normal vector of the plane created by t, facing away from the origin
    vector3 u_1 = v_sub(t.point_2, t.point_1);
    vector3 u_2 = v_sub(t.point_3, t.point_1);
    vector3 n_1 = v_cross(u_1, u_2);
    vector3 n_2 = v_normalise(n_1);
    vector3 n;
    if (v_dot(n_2, t.point_1) > 0) {
        n = n_2;
    }
    else {
        n = v_mul(n_2, -1);
    }
    //Get d, closest point of the plane from the origin (or 'distance')
    double d_param = v_dot(t.point_1, n);
    vector3 d = v_mul(n, d_param);
```

```
37        //Verify if the plane is in front of the ray or not
38        if (v_dot(v, n) < 0.001) {
39            return false;
40        }
41        //Calculate p_param
42        double a = v_dot(d, n);
43        double b = v_dot(v, n);
44        double p_param = a / b;
45        vector3 p = v_mul(v, p_param);
46        //Verify if p is in the triangle
47        vector3 u_12 = v_sub(t.point_2, t.point_1);
48        vector3 u_23 = v_sub(t.point_3, t.point_2);
49        vector3 u_31 = v_sub(t.point_1, t.point_3);
50        vector3 p_1 = v_sub(p, t.point_1);
51        vector3 p_2 = v_sub(p, t.point_2);
52        vector3 p_3 = v_sub(p, t.point_3);
53        double k_1 = v_dot(n, v_cross(u_12, p_1));
54        double k_2 = v_dot(n, v_cross(u_23, p_2));
55        double k_3 = v_dot(n, v_cross(u_31, p_3));
56        return ((k_1 > 0) && (k_2 > 0) && (k_3 > 0)) || ((k_1 < 0) && (k_2 < 0) && (k_3 < 0));
57    }
58
59    double t_intersection_distance(triangle t, vector3 v) {
60        //Get n, direction of the normal vector of the plane created by t, facing away from the origin
61        vector3 u_1 = v_sub(t.point_2, t.point_1);
62        vector3 u_2 = v_sub(t.point_3, t.point_1);
63        vector3 n_1 = v_cross(u_1, u_2);
64        vector3 n_2 = v_normalise(n_1);
65        vector3 n;
66        if (v_dot(n_2, t.point_1) > 0) {
67            n = n_2;
68        }
69        else {
70            n = v_mul(n_2, -1);
71        }
72        //Get d, closest point of the plane from the origin (or 'distance')
```

```
73          double d_param = v_dot(t.point_1, n);
74          vector3 d = v_mul(n, d_param);
75          //Get p_param, the scalar that verifies p = p_param . v
76          double a = v_dot(d, n);
77          double b = v_dot(v, n);
78          double p_param = a / b;
79          return p_param;
80      }
81
82      vector3 t_intersection_normal(triangle t, vector3 v) {
83          vector3 e1 = v_sub(t.point_2, t.point_1);
84          vector3 e2 = v_sub(t.point_3, t.point_1);
85          vector3 n1 = v_normalise(v_cross(e1, e2));
86          if (v_dot(n1, v) < 0) {
87              return n1;
88          }
89          else {
90              return v_mul(n1, -1);
91          }
92      }
```

```c
#include <assert.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

#include "vector3.h"

vector3 v_create(double x, double y, double z) {
    vector3 v = {.x = x, .y = y, .z = z};
    return v;
}

double v_norm(vector3 v) {
    return sqrt(pow(v.x, 2.0) + pow(v.y, 2.0) + pow(v.z, 2.0));
}

vector3 v_normalise(vector3 v) {
    double norm = v_norm(v);
    vector3 res = {.x = v.x / norm, .y = v.y / norm, .z = v.z / norm};
    return res;
}

vector3 v_rotate(vector3 v, double x_rot, double y_rot, double z_rot) {
    //Only for y_rot
    double new_x = v.x * cos(y_rot) - v.z * sin(y_rot);
    double new_y = v.y;
    double new_z = v.x * sin(y_rot) + v.z * cos(y_rot);
    return v_create(new_x, new_y, new_z);
}

vector3 v_add(vector3 v1, vector3 v2) {
    vector3 v_out;
```

```
37      v_out.x = v1.x + v2.x;
38      v_out.y = v1.y + v2.y;
39      v_out.z = v1.z + v2.z;
40      return v_out;
41  }
42
43  vector3 v_sub(vector3 v1, vector3 v2) {
44      vector3 v_out;
45      v_out.x = v1.x - v2.x;
46      v_out.y = v1.y - v2.y;
47      v_out.z = v1.z - v2.z;
48      return v_out;
49  }
50
51  vector3 v_mul(vector3 v, double t) {
52      vector3 v_out;
53      v_out.x = v.x * t;
54      v_out.y = v.y * t;
55      v_out.z = v.z * t;
56      return v_out;
57  }
58
59  double v_dot(vector3 v1, vector3 v2) {
60      return v1.x * v2.x + v1.y * v2.y + v1.z * v2.z;
61  }
62
63  vector3 v_cross(vector3 v1, vector3 v2) {
64      vector3 v_out;
65      v_out.x = v1.y * v2.z - v1.z * v2.y;
66      v_out.y = v1.z * v2.x - v1.x * v2.z;
67      v_out.z = v1.x * v2.y - v1.y * v2.x;
68      return v_out;
69  }
70
71  void v_print(vector3 v) {
72      printf("Vector3: %lf %lf %lf\n", v.x, v.y, v.z);
```

```
73  }
74
75  vector3 v_min(vector3* tab, int n_tab) {
76      double min_x = tab[0].x;
77      double min_y = tab[0].y;
78      double min_z = tab[0].z;
79      for (int i = 0; i < n_tab; i++) {
80          if (tab[i].x < min_x) {
81              min_x = tab[i].x;
82          }
83          if (tab[i].y < min_y) {
84              min_y = tab[i].y;
85          }
86          if (tab[i].z < min_z) {
87              min_z = tab[i].z;
88          }
89      }
90      return v_create(min_x, min_y, min_z);
91  }
92
93  vector3 v_max(vector3* tab, int n_tab) {
94      double max_x = tab[0].x;
95      double max_y = tab[0].y;
96      double max_z = tab[0].z;
97      for (int i = 0; i < n_tab; i++) {
98          if (tab[i].x > max_x) {
99              max_x = tab[i].x;
100         }
101         if (tab[i].y > max_y) {
102             max_y = tab[i].y;
103         }
104         if (tab[i].z > max_z) {
105             max_z = tab[i].z;
106         }
107     }
108     return v_create(max_x, max_y, max_z);
```

```
109     }
```